



**University of Ha'il**

**College of Computer Science and Engineering  
Department of Information Security**

---

**GRADUATION PROJECT — FINAL REPORT**

---

# **CloudAtk-ML**

## **Attack Detection in Cloud Computing Using Machine Learning**

**Under Supervision of:  
Dr. Ehab Tawfeek Alnfrawy**

**Submitted By:**

| <b>Full Name</b>               | <b>Student ID</b> |
|--------------------------------|-------------------|
| <b>Muath Salh Alharbi</b>      | <b>202206368</b>  |
| <b>Fraih Sayil Al-Shammari</b> | <b>202202917</b>  |
| <b>Omar Ahmad Al-Rashdan</b>   | <b>202200678</b>  |
| <b>Ibrahim Asm Al-Harbi</b>    | <b>202204736</b>  |
| <b>Ayman Taleb Al-Salimi</b>   | <b>202103845</b>  |

**Academic Year 2025 – 2026**

---

## Abstract

The accelerated adoption of cloud computing across enterprises and public services has introduced a parallel surge in security threats — including Denial of Service, Reconnaissance, Exploits, Backdoors, Fuzzers, and Generic attacks — that traditional signature-based defenses struggle to identify, particularly when faced with zero-day variants. As organizations continue to migrate sensitive workloads into virtualized, multi-tenant environments, attackers have correspondingly extended their toolkits to exploit weaknesses unique to cloud platforms: hypervisor escape, cross-tenant leakage, insecure management APIs, and adaptive distributed denial-of-service campaigns. The defensive technologies most widely deployed today were not designed with these threats in mind, and the gap between offensive and defensive capability has widened to a point where a fundamentally new approach is required.

This project, titled CloudAtk-ML (Cloud Defense System), addresses this gap by designing and implementing a complete machine-learning-based attack detection platform for cloud environments. The platform is not a prototype — it is a fully deployable Flask web application that ingests network features, processes them through a calibrated preprocessing pipeline, classifies them with a trained XGBoost model, persists results in a normalized MySQL schema, and presents predictions to a security administrator through a polished, multi-page dashboard. The system is engineered as a four-tier architecture (presentation, application, machine-learning, and data) so that any single layer can evolve without forcing changes in the others.

Six classification algorithms were trained and benchmarked on the UNSW-NB15 dataset, the most widely cited modern benchmark for network-attack detection: Logistic Regression, Gaussian Naive Bayes, Decision Tree, Random Forest, XGBoost, and Isolation Forest. After preprocessing (handling missing values, label-encoding categorical attributes, and applying a StandardScaler to numeric ones), models were evaluated using accuracy, precision, recall, and F1-score across a stratified 80/20 train-test split. Random Forest achieved the highest accuracy at 96.03%, followed closely by XGBoost at 95.94%, with the Decision Tree reaching 94.91% and Logistic Regression 93.58%. Confusion matrices for the two top supervised models confirm low false-negative rates against the attack class, an essential property for a security-critical system. The XGBoost model was selected as the primary engine because of its excellent balance between predictive power, training speed, and interpretability via SHAP (SHapley Additive exPlanations) values.

---

The trained models were deployed inside a full-stack web application built with Python, Flask, and MySQL. The CloudAtk-ML system delivers a security-administrator dashboard with key-performance indicators, live detection of submitted flows, deep analysis with SHAP feature impact, threat-intelligence forecasting over a seven-day horizon, a global attack map, an attack encyclopedia, attack history with PDF and CSV export, a real-time network monitor, and a security-score module. Every prediction is accompanied by a confidence value and, on demand, by a per-feature explanation, so analysts never have to trust the model blindly. The result is a deployable, scalable, and explainable cloud-defense solution that translates academic experiments into a usable product capable of supporting Saudi Arabia's digital-transformation agenda under Vision 2030.

*Keywords: Cloud Computing, Attack Detection, Machine Learning, XGBoost, Random Forest, UNSW-NB15, Intrusion Detection System, Flask, SHAP, Confusion Matrix, Cybersecurity, Saudi Vision 2030.*

---

## Acknowledgments

All praise and thanks are due to Allah, the Most Gracious, the Most Merciful, who granted us the strength, patience, and guidance to complete this graduation project. Without His help, none of the long nights of coding, the difficult debugging sessions, or the careful writing of this report would have been possible.

We extend our deepest gratitude to our supervisor, Dr. Ehab Tawfeek Alnfrawy whose continuous mentorship, technical insight, and constructive feedback shaped every chapter of this work. From the earliest sketches of the project plan to the final integration of the SHAP explainability layer, his commitment to academic excellence inspired us to push beyond a textbook implementation and build a full, working product. His weekly meetings were not only review sessions but genuine learning experiences, and we are fortunate to have benefited from his expertise.

We thank the Department of Computer Science and Information and the College of Computer Science and Engineering at the University of Hail for providing the academic environment, laboratory resources, and faculty expertise that made this project possible. We also recognize the influence of the courses we took in machine learning, software engineering, and cybersecurity — each one contributed concepts and skills that we drew on throughout the development of CloudAtk-ML.

We are also grateful to the Australian Centre for Cyber Security at the University of New South Wales for making the UNSW-NB15 dataset publicly available — without it, the empirical foundation of this work would not have been possible. We further acknowledge the open-source communities behind Python, scikit-learn, XGBoost, Flask, MySQL, and SHAP for the years of work invested in tools that we relied on every day of this project.

Finally, we thank our families for their unwavering support and patience throughout long nights of coding, debugging, and writing. Their encouragement carried us across the finish line, and their pride in our work is the most meaningful reward we could have received.

---

## Table of Contents

|                                                             |    |
|-------------------------------------------------------------|----|
| Abstract .....                                              | 2  |
| Acknowledgments .....                                       | 4  |
| Table of Contents .....                                     | 5  |
| Table of Figures .....                                      | 10 |
| Table of Tables .....                                       | 12 |
| List of Abbreviations .....                                 | 13 |
| Introduction .....                                          | 17 |
| 1. Problem Domain .....                                     | 19 |
| 1.1 Problem Statement .....                                 | 19 |
| 1.2 Proposed System .....                                   | 20 |
| 1.3 Application Domain .....                                | 21 |
| 1.4 Objectives of the Proposed System .....                 | 21 |
| 1.5 Motivation .....                                        | 22 |
| 2. Relevant Work .....                                      | 24 |
| 2.1 What is Cloud Computing? .....                          | 24 |
| 2.1.1 Cloud Service Models .....                            | 24 |
| 2.1.2 Cloud Deployment Models .....                         | 25 |
| 2.1.3 Cloud Components .....                                | 25 |
| 2.2 History of Attack Detection .....                       | 25 |
| 2.3 Overview of Current Approaches .....                    | 26 |
| 2.3.1 Common Cloud Attacks .....                            | 26 |
| 2.3.2 Machine Learning Categories .....                     | 27 |
| 2.3.3 Related Work .....                                    | 28 |
| 3. Feasibility Analysis .....                               | 30 |
| 3.1 Introduction .....                                      | 30 |
| 3.2 Technical Feasibility .....                             | 30 |
| 3.2.1 Risk regarding familiarity with the application ..... | 30 |
| 3.2.2 Risk regarding familiarity with technology .....      | 31 |
| 3.2.3 Risk regarding project size .....                     | 31 |
| 3.2.4 Risk regarding compatibility of systems .....         | 31 |
| 3.3 Cost Benefit Feasibility .....                          | 31 |
| 3.3.1 Cost Benefit Identification .....                     | 31 |

---

|                                                   |    |
|---------------------------------------------------|----|
| 3.3.2 Cost Benefit Analysis .....                 | 32 |
| 3.4 Organizational Feasibility .....              | 32 |
| 3.5 Schedule Feasibility .....                    | 33 |
| 3.5.1 Risks regarding Human Resources .....       | 33 |
| 3.5.2 Risks regarding Time .....                  | 33 |
| 3.5.3 Risks regarding Complexity .....            | 33 |
| 4. Project Management .....                       | 35 |
| 4.1 Introduction.....                             | 35 |
| 4.2 Project Methodology.....                      | 35 |
| 4.3 Project Planning .....                        | 36 |
| 5. Risk Management .....                          | 37 |
| 5.1 Introduction.....                             | 37 |
| 5.2 Risk Identification.....                      | 37 |
| 5.3 Risk Analysis .....                           | 37 |
| 5.4 Risk Planning.....                            | 38 |
| 5.5 Risk Monitoring .....                         | 39 |
| 6. Requirements Analysis .....                    | 40 |
| 6.1 Introduction.....                             | 40 |
| 6.2 Requirements Gathering .....                  | 40 |
| 6.2.1 Interviews.....                             | 40 |
| 6.2.2 Questionnaires.....                         | 40 |
| 6.2.3 Document Analysis.....                      | 41 |
| 6.3 Human-Computer Interaction Requirements ..... | 41 |
| 6.3.1 Human-Computer Interaction .....            | 41 |
| 6.3.2 Getting to know the User .....              | 41 |
| 6.3.3 Getting to know the Task.....               | 42 |
| 6.4 Requirements Specification .....              | 42 |
| 6.4.1 System Perspective .....                    | 42 |
| 6.4.2 Functional Requirements .....               | 42 |
| 6.4.3 Non-Functional Requirements .....           | 43 |
| 6.4.4 Usability Requirements.....                 | 44 |
| 7. Functional Modelling – Use Cases .....         | 45 |
| 7.1 Introduction.....                             | 45 |
| 7.2 Use-Case Diagrams.....                        | 45 |

---

|                                                                  |    |
|------------------------------------------------------------------|----|
| 7.3 Detailed Use-Case Diagrams .....                             | 46 |
| Use Case UC-04: Perform Live Detection.....                      | 47 |
| Use Case UC-05: Run Deep Analysis .....                          | 47 |
| Use Case UC-08: Review Attack History.....                       | 48 |
| 8. Behavioural Modelling – Sequence Diagram .....                | 49 |
| 8.1 Introduction.....                                            | 49 |
| 9.2 Sequence Diagrams.....                                       | 49 |
| 9.3 Concluding Object-Oriented Analysis & Design Activities..... | 52 |
| 10. Database Management and Design .....                         | 53 |
| 10.1 Introduction.....                                           | 53 |
| 10.2 Identifying Data Requirements .....                         | 53 |
| 10.3 Normalising Data .....                                      | 53 |
| 10.4 Mapping Objects to Tables .....                             | 54 |
| 10.5 Use of Indexing.....                                        | 54 |
| 10.6 Estimating Data Storage Requirements .....                  | 55 |
| 11. User-Interface Design .....                                  | 56 |
| 11.1 Introduction.....                                           | 56 |
| 11.2 Screen Design .....                                         | 56 |
| 11.2.1 Home Page (Landing Interface).....                        | 56 |
| 11.2.2 Login Interface.....                                      | 57 |
| 11.2.3 Main Dashboard.....                                       | 58 |
| 11.2.4 Global Threat Intelligence Map .....                      | 59 |
| 11.2.5 Threat Intelligence Center.....                           | 60 |
| 11.2.6 Attack History Page .....                                 | 61 |
| 11.2.7 Real-time Network Monitor.....                            | 61 |
| 11.2.8 Deep Analysis with SHAP Feature Impact.....               | 62 |
| 11.3 Visibility of System Status.....                            | 63 |
| 11.4 Match between the system and the real world .....           | 63 |
| 11.5 User control and freedom.....                               | 64 |
| 11.6 Consistency and standards .....                             | 64 |
| 11.7 Error Prevention.....                                       | 64 |
| 11.8 Recognition rather than recall.....                         | 64 |
| 11.9 Flexibility and efficiency of use .....                     | 64 |
| 11.10 Aesthetic and minimalist design .....                      | 65 |

---

|                                                                    |    |
|--------------------------------------------------------------------|----|
| 11.11 Help users recognise, diagnose and recover from errors ..... | 65 |
| 11.12 Help and documentation .....                                 | 65 |
| 12. Architectural Design .....                                     | 66 |
| 12.1 Introduction.....                                             | 66 |
| 12.2 Developing for the Web.....                                   | 66 |
| 12.2.1 Web Applications.....                                       | 66 |
| 12.2.2 Limitations of Static Websites .....                        | 66 |
| 12.2.3 Introducing Dynamic Server-Side Websites.....               | 66 |
| 12.3 Software and Hardware Considerations.....                     | 67 |
| 12.3.1 PHP vs ASP vs JSP vs Perl vs Python.....                    | 67 |
| 12.3.2 Server Combinations (Hardware) .....                        | 67 |
| 12.4 Designing the Architecture .....                              | 67 |
| 12.4.1 Limitations of a Two-Tier Architecture.....                 | 67 |
| 12.4.2 Applying a Three-Tier Architecture .....                    | 67 |
| 12.4.3 Benefits of the Three-Tier Architecture.....                | 69 |
| 12.5 Performance Evaluation Results .....                          | 69 |
| 13. Implementation Issues .....                                    | 75 |
| 13.1 Introduction.....                                             | 75 |
| 13.2 Recap on Tools & Technologies used .....                      | 75 |
| 13.2.1 Python .....                                                | 75 |
| 13.2.2 Anaconda Navigator .....                                    | 76 |
| 13.2.3 Visual Studio Code .....                                    | 76 |
| 13.2.4 Flask.....                                                  | 77 |
| 13.2.5 MySQL .....                                                 | 77 |
| 13.2.6 Draw.io.....                                                | 78 |
| 13.3 Creating the User Interface .....                             | 79 |
| 13.3.1 Designing the User Interface .....                          | 79 |
| 13.3.2 Using Standards .....                                       | 79 |
| 13.3.3 Using Cascading Style Sheets.....                           | 80 |
| 13.4 Coding Issues .....                                           | 80 |
| 13.4.1 Using Client-Side Validation.....                           | 80 |
| 13.4.2 Variable and Function Naming Convention .....               | 80 |
| 13.4.3 Adopting a Generic Approach .....                           | 81 |
| 13.4.4 Keeping Class Members Private .....                         | 81 |



---

|                                                      |    |
|------------------------------------------------------|----|
| 13.4.5 Garbage Collection .....                      | 81 |
| 13.4.6 Handling Errors.....                          | 81 |
| 13.4.7 Re-using Code.....                            | 82 |
| 13.5 Database Issues .....                           | 82 |
| 13.5.1 Choosing a suitable MySQL database type ..... | 82 |
| 13.5.2 Consistency between Tables and Classes .....  | 82 |
| 13.5.3 User Authentication .....                     | 82 |
| 13.6 Object-Oriented Code Segment Examples .....     | 83 |
| 14. Evaluation and Testing .....                     | 84 |
| 14.1 Overview of Testing .....                       | 84 |
| 14.2 Class Test Specification (Unit Testing) .....   | 84 |
| 14.3 Integration Testing .....                       | 86 |
| 14.4 System Testing.....                             | 86 |
| 15. Conclusions and Future Enhancements .....        | 88 |
| 15.1 Conclusions.....                                | 88 |
| 15.2 Future Enhancements.....                        | 89 |
| References.....                                      | 90 |
| Bibliography .....                                   | 92 |

---

## Table of Figures

| Figure      | Description                                    |
|-------------|------------------------------------------------|
| Figure 2.1  | Cloud Computing Service Models                 |
| Figure 2.2  | Cloud Deployment Models                        |
| Figure 2.3  | Cloud Computing Components Architecture        |
| Figure 2.4  | Common Attack Categories in Cloud Environments |
| Figure 2.5  | Machine Learning Categories                    |
| Figure 4.1  | Project Methodology Pipeline                   |
| Figure 4.2  | UNSW-NB15 Attack Category Distribution         |
| Figure 7.1  | Use-Case Diagram of CloudAtk-ML                |
| Figure 9.1  | Sequence Diagram for Attack Detection Flow     |
| Figure 9.2  | Activity Diagram of CloudAtk-ML                |
| Figure 11.1 | Home Page (Landing Interface)                  |
| Figure 11.2 | Login Interface                                |
| Figure 11.3 | Main Dashboard                                 |
| Figure 11.4 | Global Threat Intelligence Map                 |
| Figure 11.5 | Threat Intelligence Center                     |
| Figure 11.6 | Attack History Page                            |
| Figure 11.7 | Real-time Network Monitor                      |
| Figure 11.8 | Deep Analysis with SHAP Feature Impact         |
| Figure 12.1 | Three-Tier System Architecture                 |
| Figure 12.2 | XGBoost Internal Architecture                  |
| Figure 12.3 | Performance Comparison of ML Models            |
| Figure 12.4 | XGBoost Confusion Matrix                       |
| Figure 12.5 | Logistic Regression Confusion Matrix           |
| Figure 12.6 | SHAP Feature Importance Ranking                |

---

| Figure      | Description                 |
|-------------|-----------------------------|
| Figure 13.1 | Python Programming Language |
| Figure 13.2 | Anaconda Navigator          |
| Figure 13.3 | Visual Studio Code          |
| Figure 13.4 | Flask Server                |
| Figure 13.5 | MySQL Database              |
| Figure 13.6 | Draw.io Diagramming Tool    |

---

## Table of Tables

| Table      | Description                             | Page |
|------------|-----------------------------------------|------|
| Table 2.1  | Summary of Related Work                 | 17   |
| Table 3.1  | Cost-Benefit Analysis Summary           | 22   |
| Table 3.2  | Schedule Risk Assessment                | 25   |
| Table 5.1  | Project Risk Register                   | 33   |
| Table 6.1  | Functional Requirements                 | 40   |
| Table 6.2  | Non-Functional Requirements             | 41   |
| Table 8.1  | CRC Card — User Class                   | 50   |
| Table 8.2  | CRC Card — DetectionEngine Class        | 51   |
| Table 8.3  | CRC Card — PredictionResult Class       | 52   |
| Table 12.1 | Performance Metrics of ML Models        | 69   |
| Table 13.1 | Tools and Technologies Used             | 77   |
| Table 14.1 | Unit Test Cases — Authentication Module | 84   |
| Table 14.2 | Unit Test Cases — Prediction Module     | 84   |
| Table 14.3 | Integration Test Cases                  | 85   |
| Table 14.4 | System Test Cases                       | 86   |

---

## List of Abbreviations

The following abbreviations are used throughout this report. They are listed alphabetically together with their full meanings to support the reader's reference. Whenever an abbreviation appears for the first time in the body of the report, it is also defined in place; this list is provided as a global reference for easy lookup.

| Abbreviation | Full Meaning                       |
|--------------|------------------------------------|
| <b>AI</b>    | Artificial Intelligence            |
| <b>ANN</b>   | Artificial Neural Network          |
| <b>API</b>   | Application Programming Interface  |
| <b>APT</b>   | Advanced Persistent Threat         |
| <b>ARP</b>   | Address Resolution Protocol        |
| <b>ASP</b>   | Active Server Pages                |
| <b>BPaaS</b> | Business Process as a Service      |
| <b>CaaS</b>  | Communications as a Service        |
| <b>CNN</b>   | Convolutional Neural Network       |
| <b>CPU</b>   | Central Processing Unit            |
| <b>CRC</b>   | Class Responsibility Collaboration |
| <b>CSS</b>   | Cascading Style Sheets             |
| <b>CSV</b>   | Comma-Separated Values             |
| <b>DaaS</b>  | Desktop as a Service               |
| <b>DDoS</b>  | Distributed Denial of Service      |
| <b>DL</b>    | Deep Learning                      |
| <b>DNS</b>   | Domain Name System                 |
| <b>DoS</b>   | Denial of Service                  |
| <b>DT</b>    | Decision Tree                      |
| <b>FN</b>    | False Negative                     |

| Abbreviation | Full Meaning                                   |
|--------------|------------------------------------------------|
| <b>FP</b>    | False Positive                                 |
| <b>FR</b>    | Functional Requirement                         |
| <b>GNB</b>   | Gaussian Naive Bayes                           |
| <b>GUI</b>   | Graphical User Interface                       |
| <b>HCI</b>   | Human-Computer Interaction                     |
| <b>HTML</b>  | HyperText Markup Language                      |
| <b>HTTP</b>  | HyperText Transfer Protocol                    |
| <b>IaaS</b>  | Infrastructure as a Service                    |
| <b>ICMP</b>  | Internet Control Message Protocol              |
| <b>IDS</b>   | Intrusion Detection System                     |
| <b>IF</b>    | Isolation Forest                               |
| <b>IoT</b>   | Internet of Things                             |
| <b>IP</b>    | Internet Protocol                              |
| <b>IT</b>    | Information Technology                         |
| <b>JSON</b>  | JavaScript Object Notation                     |
| <b>JSP</b>   | Java Server Pages                              |
| <b>KPI</b>   | Key Performance Indicator                      |
| <b>LR</b>    | Logistic Regression                            |
| <b>LSTM</b>  | Long Short-Term Memory                         |
| <b>ML</b>    | Machine Learning                               |
| <b>MitM</b>  | Man-in-the-Middle                              |
| <b>MLP</b>   | Multi-Layer Perceptron                         |
| <b>MSSP</b>  | Managed Security Service Provider              |
| <b>NFR</b>   | Non-Functional Requirement                     |
| <b>NIST</b>  | National Institute of Standards and Technology |

| Abbreviation | Full Meaning                                        |
|--------------|-----------------------------------------------------|
| <b>OS</b>    | Operating System                                    |
| <b>PaaS</b>  | Platform as a Service                               |
| <b>PCA</b>   | Principal Component Analysis                        |
| <b>PDF</b>   | Portable Document Format                            |
| <b>PEP</b>   | Python Enhancement Proposal                         |
| <b>PHP</b>   | Hypertext Preprocessor                              |
| <b>REST</b>  | Representational State Transfer                     |
| <b>RF</b>    | Random Forest                                       |
| <b>SaaS</b>  | Software as a Service                               |
| <b>SHAP</b>  | SHapley Additive exPlanations                       |
| <b>SIEM</b>  | Security Information and Event Management           |
| <b>SMOTE</b> | Synthetic Minority Over-sampling Technique          |
| <b>SOC</b>   | Security Operations Center                          |
| <b>SQL</b>   | Structured Query Language                           |
| <b>SSH</b>   | Secure Shell                                        |
| <b>STIX</b>  | Structured Threat Information Expression            |
| <b>SVM</b>   | Support Vector Machine                              |
| <b>TAXII</b> | Trusted Automated eXchange of Indicator Information |
| <b>TCP</b>   | Transmission Control Protocol                       |
| <b>TN</b>    | True Negative                                       |
| <b>TP</b>    | True Positive                                       |
| <b>UAT</b>   | User Acceptance Testing                             |
| <b>UI</b>    | User Interface                                      |
| <b>UML</b>   | Unified Modelling Language                          |
| <b>UNSW</b>  | University of New South Wales                       |

---

| Abbreviation   | Full Meaning               |
|----------------|----------------------------|
| <b>URL</b>     | Uniform Resource Locator   |
| <b>VM</b>      | Virtual Machine            |
| <b>XaaS</b>    | Anything as a Service      |
| <b>XGBoost</b> | eXtreme Gradient Boosting  |
| <b>XML</b>     | eXtensible Markup Language |



---

## Introduction

Cloud computing has rapidly become the dominant model for hosting modern digital services. Organizations are migrating workloads from on-premise data centers to public, private, hybrid, and multi-cloud environments because of the cost efficiencies, elasticity, and global reach those platforms offer. The shift is not incremental — it is structural. Where computing power was once an expensive capital asset that companies bought, racked, cooled, and replaced every few years, it is now a metered utility consumed on demand. With this transformation has come an equally rapid expansion of the cyberattack surface. Enterprises in finance, government, healthcare, and education are now expected to defend complex distributed systems against an evolving catalogue of threats — Denial of Service, Distributed Denial of Service, Reconnaissance, Backdoor exploits, Fuzzers, Generic injection attempts, and increasingly sophisticated zero-day vectors that cannot be matched against any pre-existing signature.

Traditional intrusion-detection mechanisms based on rule lists or signature databases have proven insufficient in this environment. They detect what is already known but fail when adversaries use polymorphic techniques or new exploitation paths. This shortcoming has motivated the cybersecurity community to adopt machine-learning-based approaches that learn behavioural patterns from network traffic and flag deviations from normality even when the specific attack has never been seen before. The promise of machine learning in this context is not that it will eliminate every threat — no defensive technology can — but that it will close the structural gap between what attackers do and what defenders can recognize. Closing this gap requires a defender that learns continuously, generalizes across attack categories, and remains transparent enough that human analysts can audit its conclusions before acting on them.

This graduation project — CloudAtk-ML: Cloud Defense System — was conceived to deliver a practical, end-to-end machine-learning solution to this problem. Rather than stopping at a Jupyter notebook with a confusion matrix, the project produces a fully deployable Flask web application that ingests network features, runs them through a trained XGBoost classifier, persists results in a MySQL database, and presents them to a security administrator through a polished dashboard with live detection, threat intelligence, attack history, real-time monitoring, deep analysis with SHAP explainability, and a global threat map. Every screen of the system was designed with the daily workflow of a Security Operations Center analyst in mind, and the application architecture was engineered to be cleanly modular so that new models, new data sources, or new visualizations can be added without rewriting the existing code.

---

The empirical backbone of the project is the UNSW-NB15 dataset, produced by the Australian Centre for Cyber Security at the University of New South Wales. This benchmark was chosen over older alternatives such as KDD-99 and NSL-KDD because it contains modern attack categories collected from realistic network traffic and reflects the kinds of behaviour current cloud defenders must contend with. Six classification algorithms — Logistic Regression, Gaussian Naive Bayes, Decision Tree, Random Forest, XGBoost, and Isolation Forest — were benchmarked on this dataset using accuracy, precision, recall, and F1-score, supplemented by confusion-matrix analysis for the two top supervised models. The Random Forest and XGBoost models reached comparable, top-tier performance above 95.9% accuracy, and XGBoost was selected as the production engine because of its training efficiency, native support for SHAP-based interpretability, and proven robustness on imbalanced cybersecurity data.

This report is organized following the standard graduation-project structure adopted by the College of Computer Science and Engineering. Part I covers the planning phase — problem statement, related work, feasibility, project management, and risk management. Part II covers the analysis phase — requirements gathering, use-case modelling, class modelling with CRC cards, and behavioural modelling. Part III covers the design phase — database design, user-interface design, and architectural design with the full performance evaluation. Part IV covers implementation issues and the complete testing strategy with unit, integration, system, and user-acceptance test cases. The report closes with conclusions and a discussion of future enhancements that would extend the scope of the work.

---

# PART I

## PROJECT PLANNING PHASE

---

### 1. Problem Domain

#### 1.1 Problem Statement

The migration of sensitive workloads to cloud infrastructure has dramatically expanded the attack surface available to malicious actors. Cloud platforms inherit every weakness of the public Internet — IP spoofing, ARP poisoning, Distributed Denial of Service, Man-in-the-Middle interception — and add new ones unique to virtualized, multi-tenant environments: hypervisor escape, cross-tenant data leakage, insecure management APIs, browser-based control-plane vulnerabilities, and SQL or code injection through exposed services. Each of these vulnerability classes has been exploited in real-world breaches in the last several years, and the financial, reputational, and regulatory consequences for the affected organizations have been severe.

The defenses currently deployed by most cloud providers and tenant organizations are still dominated by signature-based intrusion-detection systems. These systems compare incoming traffic against a static database of known attack patterns. They are fast, deterministic, and easy to maintain, but they suffer from a fundamental limitation: they cannot detect attacks they have never seen before. Zero-day exploits, polymorphic malware, and slow-rate distributed attacks routinely bypass them. Their false-negative rate against modern adversaries is unacceptable, and their inability to learn means every new attack pattern requires a manual signature update by security analysts — a process measured in days while attacks unfold in seconds. This signature-update lag is one of the most frequently cited reasons that even well-funded organizations are still breached, and it is structural rather than incidental: signatures, by their nature, are reactive.

Cloud environments compound this problem with three additional structural challenges. First, the volume of network traffic is enormous, far beyond what human analysts can inspect. A single mid-sized cloud tenant can generate billions of packets per day, and even after aggregation into flows the volume remains too large for manual review. Second, traffic is highly heterogeneous: a single tenant may run web services, batch analytics, container orchestration, and IoT telemetry on the same infrastructure, making any fixed notion of "normal" traffic unreliable. A pattern that is benign for one workload may be suspicious for

---

another. Third, attackers themselves now use machine learning to probe and adapt — a defender that does not also learn is structurally outmatched. Adversarial techniques that intentionally evade static rules have become commoditized, available through open-source toolkits, and routinely used in observed campaigns.

The core problem this project addresses can therefore be stated as follows: traditional cloud-security mechanisms cannot reliably detect modern, evolving, and previously-unseen attacks at the speed and scale that cloud environments demand. A new approach is required — one that learns from historical traffic, generalizes to new attack patterns, runs continuously in real time, presents its findings to security administrators in an interpretable form, and integrates cleanly with the day-to-day operations of a cloud-defense team. This project sets out to design, implement, and validate exactly such an approach, producing not only a model but a complete, deployable system that an analyst could use on day one.

## 1.2 Proposed System

To address the problem stated above, this project proposes CloudAtk-ML — a machine-learning-based attack detection system delivered as a complete, deployable web application. The system is designed around four central commitments that distinguish it from the typical academic prototype and that we believe are necessary for any cybersecurity tool that aspires to real-world adoption. The first commitment is learning-based detection, the second is multi-model evaluation, the third is operational completeness, and the fourth is explainability. These four commitments shape every architectural and design decision made throughout the rest of this report.

- **Learning-based detection:** the system relies on supervised classification models trained on the UNSW-NB15 dataset rather than on static signatures, allowing it to detect both known and novel attack categories. The system is therefore not just a faster signature engine — it is a different kind of detector that learns boundaries instead of memorising fingerprints.
- **Multi-model evaluation:** six algorithms (Logistic Regression, Gaussian Naive Bayes, Decision Tree, Random Forest, XGBoost, Isolation Forest) are trained, benchmarked, and exposed to the user so that decisions are grounded in evidence rather than in a single black box. The Deep Analysis page lets the analyst compare model verdicts and confidences side-by-side.
- **Operational completeness:** the system is not a research artifact. It includes user authentication, a security-administrator dashboard, live detection, real-time monitoring, attack history with PDF/CSV export, threat intelligence forecasting, a global attack map, an attack encyclopedia, a

---

security-score module, and configurable settings. Together these modules cover the full operational lifecycle of a SOC analyst.

- **Explainability:** SHAP feature-impact visualizations are built into the deep-analysis screen, so administrators can see why a particular flow was classified as an attack instead of simply receiving an opaque verdict. Explainability is not a luxury in security — it is the basis on which an analyst decides whether to act on a model's output.

### 1.3 Application Domain

The application domain of this project is cloud-based cybersecurity operations. The intended deployment context is a cloud tenant or service provider that needs continuous monitoring of its network traffic for malicious activity. Typical users of the system include security operations center (SOC) analysts, cloud administrators, IT security officers, and managed-security-service-provider (MSSP) staff. The system has been built with the security needs of Saudi Arabian organizations in mind — particularly government agencies, banks, and healthcare providers operating under the Saudi Vision 2030 digital-transformation programme — but the design is general and applicable to any cloud-hosted environment, public or private.

The system can be deployed on a single server inside a private network or scaled horizontally behind a load balancer for larger workloads.

### 1.4 Objectives of the Proposed System

The project pursues the following concrete objectives. Each was set during the planning phase, tracked throughout the semester, and reviewed against the final deliverable. Together they form a contract between what the team promised to build and what the report demonstrates was actually built:

- Develop an attack-detection system capable of identifying and responding to security threats in cloud environments using supervised machine-learning techniques.
- Design and implement a scalable, modular architecture (presentation, application, ML, and data tiers) that can adapt to dynamic cloud workloads and evolving attack patterns.
- Investigate the comparative effectiveness of six representative algorithms — Logistic Regression, Gaussian Naive Bayes, Decision Tree, Random Forest, XGBoost, and Isolation Forest — on the UNSW-NB15 benchmark.
- Evaluate the performance of each model using accuracy, precision, recall, F1-score, and confusion matrices, and select the best-performing algorithm for production use.

- 
- Integrate the trained model into a production-style Flask web application with a complete administrator dashboard, real-time monitoring, and historical reporting.
  - Provide explainability through SHAP feature-impact visualizations so that human analysts can audit and trust the model's predictions.
  - Deliver visual situational awareness through a global threat map and threat-intelligence forecasting widgets that translate raw predictions into actionable insight.
  - Validate the entire system through unit, integration, system, and user-acceptance testing to demonstrate that it meets its functional and non-functional requirements.

## 1.5 Motivation

The motivation for this project is rooted in three observations. First, the volume and sophistication of cloud-targeted attacks have grown faster than the capabilities of signature-based defenses. Public reports show year-on-year increases in cloud breaches, ransomware attacks against tenants, and DDoS campaigns against cloud-hosted services. The economic impact of these incidents — measured in lost revenue, regulatory fines, and remediation costs — runs into billions of dollars annually and is growing. Second, sensitive data — financial records, health information, government documents — is increasingly stored in cloud environments, raising the stakes of every successful breach. A single compromise can affect millions of users in a way that on-premise breaches rarely could. Third, well-publicized regulatory and reputational consequences mean that organizations can no longer treat security as an afterthought; they must demonstrate continuous, proactive monitoring.

Beyond these external drivers, the team was motivated by the academic and engineering challenge of bridging the gap that often exists between machine-learning research and real-world deployment. Many published intrusion-detection studies stop at reporting accuracy on a benchmark dataset. Few of them produce a working product that a security administrator could actually log in to, configure, and use. Our motivation was precisely to close this gap — to build something that performs well on UNSW-NB15 and that also looks, feels, and behaves like a tool ready for a real security operations center. We wanted our deliverable to be something we would be proud to demonstrate not only in front of an academic committee but also in front of a working SOC manager.

Finally, the project carries a national dimension. Saudi Arabia's Vision 2030 strategy explicitly emphasizes the development of national cybersecurity capacity and the migration of government services

---

to cloud infrastructure. A defensible cloud requires defensive technology, and that technology requires Saudi engineers trained to build it. By undertaking this project, the team contributes — at a small but meaningful scale — to the human-capital component of that national strategy. The skills we developed in machine learning, full-stack web engineering, and applied cybersecurity are exactly the skills the Kingdom is investing in for the next decade of digital transformation.

---

## 2. Relevant Work

### 2.1 What is Cloud Computing?

Cloud computing is the on-demand delivery of computing resources — servers, storage, networks, software, and analytics — over the Internet, with pricing based on actual consumption. By analogy, cloud services behave like utilities such as electricity or water: the user does not own the production infrastructure and pays only for what is used. This shift from capital expenditure to operating expenditure has transformed the economics of IT and has enabled organizations of every size to access the same class of computing power that was once available only to large enterprises with deep pockets and dedicated data centers. Cloud computing is one of the main technologies driving the way modern organizations work and play, and it has helped companies reduce IT headaches while boosting productivity and security.

Three structural advantages explain the rapid market adoption of cloud computing. First, time-to-deployment is dramatically shorter — provisioning a new server now takes minutes rather than weeks, allowing organizations to respond to new opportunities and challenges with speed that was previously inconceivable. Second, capital outlay is minimized, freeing cash flow for product investment, marketing, hiring, or research and development. Third, elasticity is intrinsic: workloads can scale up or down in near real time as demand fluctuates, so an e-commerce site that needs 10 servers on a normal day and 100 on Black Friday can provision exactly that, paying only for what is used.

#### 2.1.1 Cloud Service Models

The National Institute of Standards and Technology (NIST) recognizes three foundational service models, each of which represents a different division of responsibility between the cloud provider and the customer. Software as a Service (SaaS) is the most familiar model, where the provider hosts and manages a complete software application that the customer accesses through a web browser; examples include Gmail, Salesforce, and Microsoft 365. Platform as a Service (PaaS) sits one layer deeper: the provider supplies a managed runtime, libraries, and tooling on which the customer deploys application code without managing the underlying servers; examples include Google App Engine and AWS Elastic Beanstalk. Infrastructure as a Service (IaaS) is the most flexible model: the provider exposes virtualized compute, storage, and networking that the customer configures as if it were a private data center; examples include Amazon EC2 and Google Compute Engine.



---

### 2.1.2 Cloud Deployment Models

Independently of the service model, cloud environments are deployed in one of four ways. The deployment model determines who owns the underlying infrastructure and who has access to it. A Public Cloud is owned and operated by a third-party provider (AWS, Microsoft Azure, Google Cloud, IBM, Alibaba Cloud) and shared among many tenants — this model offers the lowest entry cost and the fastest scaling but raises shared-tenancy security questions that machine-learning detection systems are particularly suited to address. A Private Cloud is dedicated to a single organization, hosted in its own data center or in a third-party facility — this model provides the strongest control over data governance, compliance, and security, at the cost of higher capital outlay and reduced elasticity. A Hybrid Cloud combines public and private clouds connected so that data and applications can move between them — this model lets organizations keep sensitive workloads private while bursting to public capacity for elastic demand and is the dominant pattern in regulated industries. Multi-Cloud is the parallel use of services from multiple cloud providers, typically to avoid vendor lock-in, optimize cost, or comply with regional regulations.

### 2.1.3 Cloud Components

A cloud system can be decomposed into a front-end client tier and a back-end server tier connected by the Internet. Within these tiers the principal components are: the Client Infrastructure (browser, mobile app, dashboard); the Application layer where tenant software runs; the Service layer that manages SaaS, PaaS, and IaaS offerings; the Runtime Cloud where virtual machines execute concurrently; the Storage subsystem (object storage, block storage, archives); the underlying physical Infrastructure of servers, networks, and virtualization fabric; the Management plane that orchestrates resources; and the Security plane that enforces firewalls, identity, and access controls. Continuous communication across these components creates many opportunities for adversaries to interpose themselves — which is exactly what attack-detection systems must monitor. A single compromised component can ripple across all the others, which is why detection must be both broad in coverage and deep in fidelity.

## 2.2 History of Attack Detection

Attack detection has gone through three distinct generations. The first generation, beginning in the late 1980s, was based on signature matching: incoming traffic was compared against a database of known attack patterns, and any match raised an alert. Snort, the most influential open-source intrusion detection system of this era, demonstrated that signature matching could be done at line rate on commodity hardware

---

and that the resulting alerts could be useful to security analysts. However, signature-based systems suffered from two structural weaknesses: they could not detect previously unknown attacks, and they generated large numbers of false positives when signatures were too broad.

The second generation, beginning in the early 2000s, introduced anomaly detection based on statistical models of normal behaviour. A baseline of normal traffic was learned from clean data, and any traffic that deviated significantly from the baseline was flagged as potentially malicious. This approach in principle could detect novel attacks, but in practice it suffered from the difficulty of defining "normal" in environments where workloads change continuously. False-positive rates remained high, and the systems required constant retuning to remain useful. Despite these limitations, anomaly detection introduced the principle that attack detection could be data-driven rather than rule-driven, paving the way for the third generation.

The third generation, which is still emerging, applies modern machine learning and deep learning to the problem. Trained classifiers — logistic regression, decision trees, random forests, gradient-boosted trees, neural networks — learn complex decision boundaries from labeled traffic and generalize to flows they have never seen. Deep-learning architectures (CNN, LSTM, Transformer) capture sequential and structural patterns that simpler models miss. Explainability techniques such as SHAP and LIME allow human analysts to audit the reasoning behind individual predictions, addressing the long-standing concern that data-driven detection produces opaque verdicts. CloudAtk-ML belongs squarely to this third generation: it is a learning-based system, it benchmarks multiple algorithms, and it integrates SHAP-based explainability as a first-class feature.

## **2.3 Overview of Current Approaches**

Modern approaches to attack detection in cloud environments fall into three broad categories: classical machine learning, deep learning, and ensemble or hybrid methods. Each approach has strengths and trade-offs that this project carefully considered before selecting XGBoost as the production engine.

### **2.3.1 Common Cloud Attacks**

Cloud environments are exposed to a wide spectrum of attacks. The most commonly studied categories are listed below. Each one motivates one or more of the supervised-learning labels found in the UNSW-NB15 benchmark dataset and therefore shapes the predictive surface that CloudAtk-ML must cover.

- 
- **DoS / DDoS** — Denial-of-Service and Distributed Denial-of-Service attacks attempt to exhaust the resources of a target service, causing it to become unavailable to legitimate users. They are particularly damaging in cloud contexts because availability is a primary value proposition and because cloud providers may charge tenants for the spurious traffic.
  - **Botnets** — networks of compromised hosts coordinated by a remote command-and-control infrastructure. Botnets propagate themselves, exfiltrate data, and serve as the engine behind many DDoS campaigns. Modern bots can recruit other vulnerable systems autonomously and report status back to their operators.
  - **Malware** — viruses, worms, and rootkits that compromise virtual machines and persist in the cloud environment. Continuous monitoring is essential at scale because a single infected VM can become a foothold for lateral movement.
  - **Phishing** — social-engineering attempts to trick users into revealing credentials. In the cloud, phishing kits are sometimes hosted on legitimate-looking cloud services to evade reputation filters.
  - **Man-in-the-Middle** — interception of traffic between users and cloud services. The attacker sits transparently in the communication path and steals or manipulates data, often with the goal of harvesting credentials or session tokens.
  - **Injection Attacks** — SQL injection, code injection, and command injection that exploit poorly validated user input to gain unauthorized control or exfiltrate data. These attacks remain among the most common entry vectors in published breach reports.
  - **Data Breaches** — unauthorized access to stored sensitive data through exploitation of misconfigured permissions, exposed credentials, or vulnerabilities in storage services. Data breaches are the most damaging single category of cloud incident in financial terms.

### 2.3.2 Machine Learning Categories

Machine learning provides the analytical foundation for modern attack detection. Four learning paradigms are relevant, each appropriate to a different kind of available data and a different operational scenario. Supervised learning trains a model on labeled examples and is the dominant paradigm in CloudAtk-ML. Unsupervised learning works on unlabeled data and discovers structure on its own — Isolation Forest is the unsupervised baseline included in our benchmark. Semi-supervised learning combines labeled and unlabeled data, which is useful when labeling is expensive but raw traffic is

---

abundant. Reinforcement learning has an agent interact with an environment and learn an optimal policy, which is more applicable to adaptive intrusion-response than to detection per se.

### 2.3.3 Related Work

Several published studies have explored machine-learning-based attack detection in cloud environments. The most relevant are summarized below; together they provide both methodological precedent and a benchmark against which the present project can be compared.

He, Zhang, and Lee (2017) proposed a method for detecting DDoS attacks against cloud providers from the source side using statistical features of virtual-machine network traffic. They evaluated nine machine-learning algorithms across four DDoS subtypes — SSH brute-force, DNS reflection, ICMP flooding, and TCP SYN attacks — and reported error rates below 0.07% with accuracy approaching 99.7% in a controlled test environment. Their work demonstrated that machine learning can reach extremely high accuracy on narrow attack classes when the feature engineering is well aligned with the threat.

Chu, Lin, and Chang (2019) addressed the detection and classification of advanced persistent threats (APT) using Support Vector Machines on the NSL-KDD dataset. They compared SVM with Naive Bayes, J48 Decision Tree, and Multi-Layer Perceptron, finding that SVM produced the highest recognition rate at 97.22%. Their study confirmed that algorithm choice matters even when the dataset and feature engineering are held constant, and it provided one of the cleanest comparative benchmarks in the recent literature.

Ahsan and Nygard (2020) investigated convolutional neural networks combined with Long Short-Term Memory units for intrusion detection in wireless sensor networks, also using the NSL-KDD dataset. Their hybrid CNN-LSTM model improved detection of unknown attacks and reduced false-positive rates on minority attack classes. Their results suggest that deep-learning approaches retain value for problems with sequential or spatial structure, even when classical machine learning is competitive on the global accuracy metric.

Tiwari and Jain (2022) proposed a new firewall mechanism that uses machine learning and deep learning to classify incoming traffic in a secure cloud environment. They introduced a synthetic ensemble approach called "Most Frequent Decision" that concatenates the outputs of several base classifiers and reported a detection rate above 90% and a per-class classification rate above 80% on the UNSW-NB15 dataset —

the same dataset adopted in the present project. Their results form a direct point of comparison for our own benchmark numbers and motivated our decision to test multiple classifiers in parallel rather than relying on a single algorithm.

| Reference             | Technique                                           | Dataset                       | Key Findings                                                                                          |
|-----------------------|-----------------------------------------------------|-------------------------------|-------------------------------------------------------------------------------------------------------|
| He et al. (2017)      | 9 ML methods for DDoS detection                     | VM network traffic statistics | <0.07% error rate, ~99.7% accuracy on SSH, DNS reflection, ICMP, TCP SYN attacks                      |
| Chu et al. (2019)     | SVM for APT detection                               | NSL-KDD                       | SVM reached 97.22% accuracy, outperforming Naive Bayes, J48, and MLP                                  |
| Ahsan & Nygard (2020) | Hybrid CNN-LSTM                                     | NSL-KDD                       | Improved detection of unknown attacks, reduced false positives on minority classes                    |
| Tiwari & Jain (2022)  | ML/DL firewall with Most Frequent Decision ensemble | UNSW-NB15                     | >90% detection rate, >80% per-class classification rate                                               |
| This Project (2024)   | 6 ML models with XGBoost as production engine       | UNSW-NB15                     | Random Forest 96.03%, XGBoost 95.94%, deployed in full Flask web application with SHAP explainability |

*Table 2.1: Summary of Related Work*

---

## 3. Feasibility Analysis

### 3.1 Introduction

Feasibility analysis evaluates whether a proposed project is realistic from technical, economic, organizational, and scheduling perspectives. For CloudAtk-ML, this analysis was conducted before development began and revisited at the end of each milestone. The four dimensions assessed below collectively confirmed that the project was achievable within the timeline and resources available to the team. Each dimension is treated separately because each carries different kinds of risk: technical risk concerns whether the team can build the system at all; economic risk concerns whether the costs are justified; organizational risk concerns whether the system fits the institution that will use it; and scheduling risk concerns whether the work can be completed in time. The analysis below was endorsed by the project supervisor at the end of the planning phase and served as the formal go-ahead for the implementation phase.

### 3.2 Technical Feasibility

Technical feasibility addresses whether the team has the skills and the technology to deliver the system. We broke this question into four sub-categories of risk and assessed each one in turn. The overall technical risk was judged to be medium, with all major risks having identified mitigation strategies before implementation began.

#### 3.2.1 Risk regarding familiarity with the application

All team members had prior coursework in cybersecurity fundamentals and had been exposed to intrusion-detection concepts. The risk of misunderstanding the application domain was therefore low. Domain knowledge gaps were addressed through guided reading of the related-work studies summarized in Chapter 2 and through weekly meetings with the project supervisor. We also reviewed published documentation on the UNSW-NB15 dataset to ensure that we understood the meaning of every feature column before we began modelling. Whenever a domain question arose during implementation that could not be answered internally, we either consulted the supervisor directly or tabled the question for the next weekly meeting.

---

### **3.2.2 Risk regarding familiarity with technology**

The technical stack — Python, scikit-learn, XGBoost, Flask, MySQL, HTML/CSS/JavaScript, Bootstrap, Chart.js — was largely covered in earlier courses. The two areas with a steeper learning curve were XGBoost hyperparameter tuning and SHAP-based explainability; both were mitigated by pair programming and dedicated study sessions. We also leaned on the abundant open-source documentation and tutorials available for each tool. Risk was assessed as medium and was successfully managed through investment in early-semester learning sprints. By week six, every team member had at least one module they were comfortable owning.

### **3.2.3 Risk regarding project size**

The project is medium-large in scope: a multi-page Flask application with sixteen HTML templates, six trained models, a relational database, and an integrated SHAP-based analysis page. Risk was managed by partitioning work along clean module boundaries (authentication, dashboard, prediction, history, intelligence, monitoring) and by enforcing a sprint-based development cadence with weekly checkpoints. Each module was owned by a primary developer with a designated reviewer, ensuring that no single piece of code was understood by only one person. The CRC cards described later in Chapter 8 made this partition explicit.

### **3.2.4 Risk regarding compatibility of systems**

All chosen technologies are open-source and cross-platform. The XGBoost library, Flask web framework, and MySQL database run on Windows, Linux, and macOS without modification, so portability was not a concern. The team standardized on Python 3.10+ to avoid version incompatibility, and dependency versions were pinned in a shared requirements file. Risk was low and remained low throughout the semester. The single compatibility issue encountered — an older NumPy version conflicting with newer SHAP — was resolved within the first sprint by pinning NumPy to a compatible release.

## **3.3 Cost Benefit Feasibility**

### **3.3.1 Cost Benefit Identification**

Because the project is academic and uses only open-source tooling, direct monetary costs are negligible. The principal investments are time and computational effort. The benefits, however, extend beyond grading: the resulting system can be re-purposed in real security operations centers and contributes to the

broader cyber-defense capacity of the Kingdom of Saudi Arabia under Vision 2030. Below we identify each cost and benefit in turn before tabulating them.

### 3.3.2 Cost Benefit Analysis

Table 3.1 summarizes the cost-benefit calculation. The dominant cost is human effort — about 700 person-hours total across the team — and the dominant benefit is the production of a reusable, deployable security tool that demonstrates real cybersecurity engineering capability.

| Item                           | Description / Estimated Value                                                                                           |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Software costs</b>          | 0 SAR — all tools used (Python, Flask, MySQL, XGBoost, scikit-learn, VS Code, Anaconda) are open source.                |
| <b>Hardware costs</b>          | 0 SAR additional — development used team members' existing laptops.                                                     |
| <b>Dataset licensing</b>       | 0 SAR — UNSW-NB15 is freely distributed for academic research.                                                          |
| <b>Effort cost (estimated)</b> | $\approx 700$ person-hours across 5 students $\times \approx 140$ hours each over a single semester.                    |
| <b>Tangible benefits</b>       | Working defensive system that can detect cloud attacks at $>95\%$ accuracy; reusable code base.                         |
| <b>Intangible benefits</b>     | Hands-on team experience with ML, MLOps, and full-stack development; contribution to national cyber-defense literature. |

*Table 3.1: Cost-Benefit Analysis Summary*

### 3.4 Organizational Feasibility

Organizational feasibility asks whether the system fits the goals of the institution that will use it. CloudAtk-ML aligns with the Saudi Vision 2030 National Cybersecurity Strategy, with the Kingdom's drive toward cloud-first government services, and with the College of Computer Science and Engineering's emphasis on applied capstone projects. The system is therefore organizationally feasible from both a national and an academic perspective. The team also confirmed with the supervisor that the chosen scope matched the academic expectations of the College for a Graduation Project I deliverable. Beyond the institutional fit, the project also fits the broader trajectory of the cybersecurity industry, which is moving rapidly toward learning-based detection and explainability — meaning the skills the team developed are directly transferable to industry roles after graduation.



---

## 3.5 Schedule Feasibility

### 3.5.1 Risks regarding Human Resources

The team comprises five members with overlapping but complementary skills: machine learning, web development, database design, UI/UX, and documentation. The risk of a single point of failure was minimized through cross-training so that every module had at least two contributors familiar with it. Weekly meetings ensured that absences could be covered without the project losing momentum. We also adopted a shared issue tracker and code repository so that any team member could pick up a task left incomplete by another.

### 3.5.2 Risks regarding Time

The project was delivered within a single semester. Time risk was mitigated by a phased plan: weeks 1–2 for problem definition and dataset acquisition, weeks 3–6 for model training and evaluation, weeks 7–11 for web-application development, weeks 12–13 for integration and testing, and weeks 14–15 for documentation and presentation. Each phase ended with a tangible deliverable that could be inspected by the supervisor and incorporated into the final report. Slack time was built into each phase to absorb unexpected delays without putting the final delivery date at risk.

### 3.5.3 Risks regarding Complexity

The principal complexity-related risks were the integration of XGBoost into Flask (resolved using joblib serialization), the deployment of SHAP explainability without slowing down the prediction endpoint (resolved by pre-computing SHAP values for sample inputs), and the design of a coherent UI across sixteen pages (resolved by adopting a single shared Bootstrap-based layout template). Each risk was tracked in the team's risk register and reviewed every week until it was either retired or had its mitigation confirmed. Complexity risk was the most active category throughout the semester but was managed through disciplined scope control.

---

| Risk Category                               | Likelihood | Mitigation                                                                                    |
|---------------------------------------------|------------|-----------------------------------------------------------------------------------------------|
| Team-member illness or absence              | Medium     | Cross-training; weekly knowledge transfer; shared Git repository.                             |
| Underestimated implementation effort        | Medium     | Sprint-based plan with 1-week buffers; scope flexibility on "nice-to-have" features.          |
| Model under-performance on minority classes | High       | Multi-model benchmarking; class-weighted training; SMOTE if needed.                           |
| Integration delays between ML and web layer | Medium     | Early integration spike; well-defined REST API contracts; mock data for parallel development. |

*Table 3.2: Schedule Risk Assessment*

---

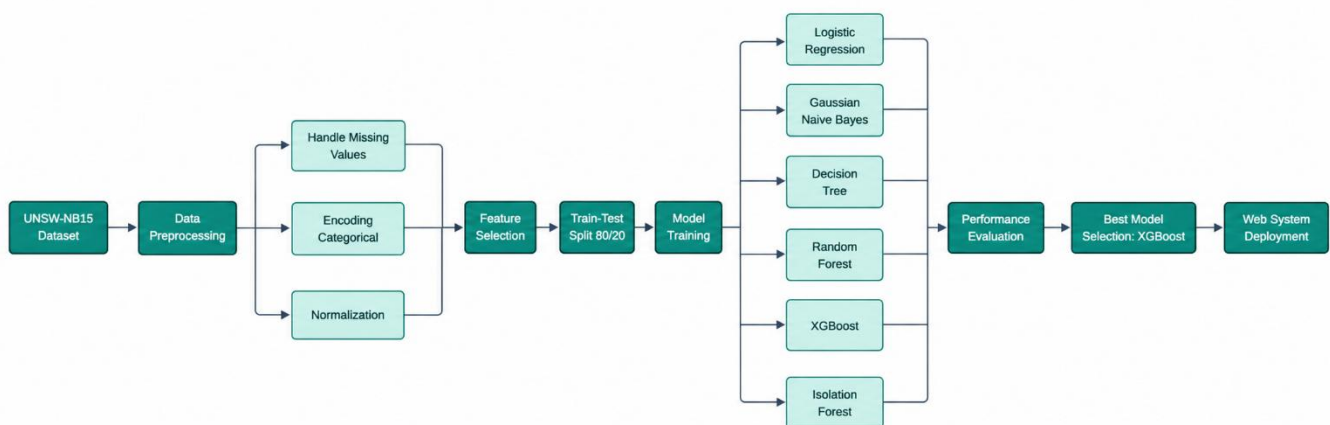
## 4. Project Management

### 4.1 Introduction

Project management governs how the work of building CloudAtk-ML is planned, scheduled, executed, and controlled. The team adopted an agile-inspired iterative methodology, complemented by a Gantt-style high-level plan, and held weekly meetings with the supervisor to review progress and adjust scope. Project management was treated as a first-class activity rather than an afterthought, because experience from past student projects suggested that lack of structure was a more common failure mode than lack of skill. The framework described in this chapter provided a backbone of accountability that kept the project on schedule and aligned with the supervisor's expectations from the first week to the last.

### 4.2 Project Methodology

The methodology of CloudAtk-ML is summarized in two figures. Figure 4.1 shows the high-level methodology adopted at the start of the project, beginning with cloud-computing data, passing through the UNSW-NB15 dataset for analysis and preprocessing, and ending with the comparison of four models and the selection of XGBoost as the production model. Figure 4.2 shows the more detailed pipeline that we ultimately implemented, expanded to six models. The pipeline is intentionally linear at the model-training stage and parallel across the candidate classifiers, allowing direct comparison under identical preprocessing conditions.



*Figure 4.1: Project Methodology Pipeline*

---

### 4.3 Project Planning

The semester-long timeline was divided into five phases. The first phase (weeks 1–2) was dedicated to problem definition, supervisor alignment, and dataset acquisition. The second phase (weeks 3–6) concentrated on data preprocessing, exploratory data analysis, and the training of the six baseline models. The third phase (weeks 7–11) focused on full-stack web development — Flask routes, MySQL schema, sixteen HTML templates, Bootstrap styling, Chart.js visualizations, and authentication. The fourth phase (weeks 12–13) covered system integration, end-to-end testing, and SHAP integration on the deep-analysis page. The fifth phase (weeks 14–15) was reserved for final documentation, the preparation of this report, and the project presentation.

Within each phase, work was tracked in a shared issue board. Tasks were estimated in story points, and the team held a 30-minute stand-up twice a week to surface blockers early. The weekly meeting with the supervisor served as the formal milestone review. This rhythm ensured that no single week passed without measurable progress and without an external check on the direction of the work. The combination of agile sprints internally and waterfall-style milestone reviews externally gave us the best of both planning approaches: flexibility on a daily basis and predictability on a weekly basis.

---

## 5. Risk Management

### 5.1 Introduction

Risk management is the systematic identification, analysis, and mitigation of factors that could threaten project success. For CloudAtk-ML, risks were tracked in a shared risk register and reviewed at every weekly meeting. Risk management was treated as a continuous process rather than a one-time exercise, because we knew from coursework that the most damaging risks are usually the ones that emerge late and unnoticed. The four sub-sections below — identification, analysis, planning, and monitoring — cover the full risk-management lifecycle and explain how each step was implemented in practice during the semester.

### 5.2 Risk Identification

The team identified the following primary risks early in the planning phase. Each risk was assigned an owner, a severity rating, and an explicit mitigation plan. The risks below were not chosen randomly — they were drawn from a survey of past graduation projects, from discussions with the supervisor, and from a brainstorming session held in the second week of the semester.

- Data-quality risk: the UNSW-NB15 dataset is large but contains class imbalance that could bias models toward majority classes.
- Model-performance risk: a single algorithm may not generalize across all attack categories.
- Integration risk: serializing trained models and loading them efficiently inside Flask.
- Security risk: storing user credentials safely in the MySQL database.
- Scope-creep risk: feature ideas accumulating during development beyond what time allowed.
- Documentation risk: leaving report writing to the final two weeks.

### 5.3 Risk Analysis

Each identified risk was assessed for likelihood and impact, and an explicit mitigation strategy was assigned. The risk register, summarized in Table 5.1, was the document that drove most of the project's defensive decisions: it informed the choice to benchmark six models rather than just one, the decision to begin documentation in week four rather than week fourteen, and the choice to use werkzeug's password hashing rather than a hand-rolled scheme.

| Risk                           | Likelihood | Impact | Mitigation Strategy                                                                       |
|--------------------------------|------------|--------|-------------------------------------------------------------------------------------------|
| Class imbalance in dataset     | High       | Medium | Use class-weighted training; report per-class precision/recall; consider SMOTE if needed. |
| Single model under-performance | Medium     | High   | Train and benchmark six different models; pick the strongest empirically.                 |
| Slow Flask predictions         | Medium     | Medium | Pre-load models at server startup; cache scaler and label encoder.                        |
| Insecure password storage      | Low        | High   | Hash passwords with werkzeug.security and salt; never store plaintext.                    |
| Scope creep                    | High       | Medium | Maintain frozen feature list after week 6; new ideas go to a future-work log.             |
| Late documentation             | Medium     | Medium | Begin writing chapters 1 and 2 in week 4; update report continuously.                     |

*Table 5.1: Project Risk Register*

## 5.4 Risk Planning

Risk planning translated the abstract identifications and analyses into concrete mitigation actions tied to specific people and dates. For each risk in the register, the team assigned a primary owner — usually the team member whose work was most affected by the risk — and a backup owner who would step in if the primary owner was unavailable. Each mitigation action was time-bounded: for example, the mitigation for late documentation was to begin writing chapters 1 and 2 by the end of week 4, an action that was tracked on the issue board and signed off by the supervisor at the week-4 meeting.

Beyond the per-risk plans, the team established three cross-cutting risk-planning practices. The first was a code-review requirement: every pull request had to be reviewed by at least one team member who did not write the code. The second was a backup policy: all code, models, and documentation were committed daily to a shared Git repository hosted off the team's local machines. The third was a contingency reserve of one week at the end of the semester, kept free of new feature work, that could be used to absorb any unexpected delays from the preceding fourteen weeks.

---

## 5.5 Risk Monitoring

Each identified risk was assigned to a responsible team member who reported on it during the weekly stand-up. The risk register was a living document, updated whenever a new risk was discovered or an existing one was retired. By the end of the semester, all six identified risks had either been mitigated successfully or had not materialized. This continuous monitoring approach proved valuable: it surfaced the SHAP-performance issue (originally judged a low-likelihood risk) early enough that we had time to design a solution involving cached SHAP values rather than on-demand computation. Two risks not originally identified — a brief database-connection-pool exhaustion problem in week 11 and a small CSS layout regression in week 13 — were caught quickly because the monitoring rhythm meant problems were never allowed to accumulate silently.

---

## PART II

# PROJECT ANALYSIS PHASE

---

## 6. Requirements Analysis

### 6.1 Introduction

Requirements analysis defines what the system must do (functional requirements) and the qualities it must exhibit (non-functional requirements). The CloudAtk-ML requirements were elicited from three sources: the project supervisor, the related-work studies surveyed in Chapter 2, and a structured walk-through of the typical workflow of a security-operations-center analyst. The chapter that follows describes the elicitation process, the analysis of human-computer interaction needs, and the formal specification of functional, non-functional, and usability requirements.

### 6.2 Requirements Gathering

Requirements were gathered through three complementary techniques: interviews, questionnaires, and document analysis. Each technique surfaced different kinds of information, and the combination produced a more complete picture than any single technique would have.

#### 6.2.1 Interviews

Three semi-structured interviews were conducted with the project supervisor across the planning phase. The first interview clarified the academic scope and confirmed the team's understanding of what a Graduation Project I deliverable should look like. The second interview focused on what "a complete system" should look like — i.e., the difference between a Jupyter-notebook prototype and a deployable web application. The third interview reviewed the proposed feature list and pruned items considered out of scope for a single semester. Each interview was minuted, the minutes were reviewed at the next weekly meeting, and any decisions made were captured directly in the requirements specification below.

#### 6.2.2 Questionnaires

A short questionnaire was distributed to peers in the cybersecurity track to validate priorities. The questionnaire asked respondents to rank a list of candidate features in order of usefulness for a SOC analyst. Respondents ranked "clarity of attack alerts" and "a global threat map" as the two most useful



---

features, which informed our decision to prioritize the dashboard and the attack-map page. The questionnaire also surfaced two features we had not originally considered — a security-score module and an attack encyclopedia — both of which were added to the scope based on respondent feedback.

### **6.2.3 Document Analysis**

The team reviewed academic papers (Chapter 2 references), product documentation for tools used in industry (Splunk, IBM QRadar, Microsoft Sentinel), and the schema of the UNSW-NB15 dataset. This document analysis directly shaped the feature set, the database schema, and the metric set used for model evaluation. By studying the kinds of dashboards and alerts used in production SIEM tools, the team ensured that CloudAtk-ML's UI would feel familiar to anyone with prior security-operations experience rather than appearing as an academic curiosity.

## **6.3 Human-Computer Interaction Requirements**

### **6.3.1 Human-Computer Interaction**

CloudAtk-ML is a security tool, so its UI must respect three HCI principles drawn from Nielsen's heuristics. First, the most important information must be visible without scrolling — the dashboard places KPI cards above the fold for exactly this reason. Second, high-severity alerts must be visually distinct — colour, size, and position all reinforce severity ranking. Third, actions must be reversible whenever possible — the few destructive actions in the system (delete settings, clear history) require explicit confirmation and have undo paths where feasible. These principles guided the design of the dashboard, the attack history filters, and the threat-intelligence page.

### **6.3.2 Getting to know the User**

The primary user is a security administrator who is technically literate but not necessarily a machine-learning expert. The user is expected to understand network attacks at a conceptual level, to read charts comfortably, and to operate filtering and exporting controls without training. The secondary user is a regular user who accesses the home page, the attack encyclopedia, and the global attack map for awareness purposes. Designing for both audiences without compromising either was an explicit goal: the public pages were kept clean and educational, while the authenticated pages were dense, data-rich, and oriented toward action.

---

### 6.3.3 Getting to know the Task

The principal task is continuous monitoring of cloud network traffic for malicious activity, with periodic deep-dives into individual incidents. Sub-tasks include: viewing aggregate statistics, drilling down into recent alerts, exporting reports for compliance purposes, and tuning system settings. Each sub-task corresponds to one or more pages in the application: the dashboard for aggregate stats, the attack history for drill-down, the export controls for compliance, and the settings page for tuning. The use cases described in Chapter 7 trace each of these sub-tasks through the system in detail.

## 6.4 Requirements Specification

### 6.4.1 System Perspective

CloudAtk-ML is a standalone web application that runs on a single Flask server, connects to a MySQL database, and serves a browser-based UI. It does not currently integrate with external SIEM systems, but its modular API endpoints would allow such integration in future work. The system is deployed as a single Python process behind a development web server during evaluation; for production, a WSGI server such as Gunicorn or uWSGI behind a reverse proxy would be appropriate.

### 6.4.2 Functional Requirements

Table 6.1 lists the twelve principal functional requirements of CloudAtk-ML. Each requirement is identified by a unique FR-XX code, given a short name, and accompanied by a brief description. The full breakdown of each functional requirement, including pre-conditions, post-conditions, and exception flows, is provided in Appendix [H].

| ID    | Requirement       | Description                                                                                                             |
|-------|-------------------|-------------------------------------------------------------------------------------------------------------------------|
| FR-01 | User Registration | The system shall allow a new user to create an account with username, email, and password.                              |
| FR-02 | User Login        | The system shall authenticate registered users and create a session.                                                    |
| FR-03 | Dashboard         | The system shall display total predictions, attacks detected, normal traffic count, and accuracy on a single dashboard. |
| FR-04 | Live Detection    | The system shall accept network-flow features and return a predicted label and confidence score.                        |

| ID    | Requirement         | Description                                                                                                                  |
|-------|---------------------|------------------------------------------------------------------------------------------------------------------------------|
| FR-05 | Deep Analysis       | The system shall display SHAP feature-impact values and a model-performance comparison for each prediction.                  |
| FR-06 | Threat Intelligence | The system shall compute a global threat level and a 7-day risk forecast.                                                    |
| FR-07 | Attack Map          | The system shall visualize attack origins on a global map with a live threat feed.                                           |
| FR-08 | Attack History      | The system shall list past attacks with date, type, severity, and resolution status, and allow filtering and PDF/CSV export. |
| FR-09 | Real-time Monitor   | The system shall display live traffic statistics, system status, CPU, memory, and packet rate.                               |
| FR-10 | Attack Encyclopedia | The system shall provide an educational reference describing each attack category.                                           |
| FR-11 | Security Score      | The system shall compute and display an overall security score for the monitored environment.                                |
| FR-12 | Settings            | The system shall allow administrators to configure thresholds, notifications, and account details.                           |

*Table 6.1: Functional Requirements*

### 6.4.3 Non-Functional Requirements

Non-functional requirements describe the qualities the system must exhibit rather than the actions it must perform. Table 6.2 lists the eight principal non-functional requirements.

| ID     | Quality     | Description                                                                                 |
|--------|-------------|---------------------------------------------------------------------------------------------|
| NFR-01 | Performance | Predictions shall be returned in under 500 ms on standard hardware.                         |
| NFR-02 | Reliability | The system shall have an uptime of 99% during normal operating hours.                       |
| NFR-03 | Security    | Passwords shall be stored only as salted hashes; sessions shall expire after inactivity.    |
| NFR-04 | Usability   | First-time users shall be able to complete login and dashboard navigation within 5 minutes. |
| NFR-05 | Scalability | The architecture shall support adding new models without changing the API contract.         |

| ID            | Quality                | Description                                                                    |
|---------------|------------------------|--------------------------------------------------------------------------------|
| <b>NFR-06</b> | <b>Maintainability</b> | Code shall follow PEP-8 conventions and be documented inline.                  |
| <b>NFR-07</b> | <b>Portability</b>     | The system shall run on any OS that supports Python 3.10+ and MySQL 8+.        |
| <b>NFR-08</b> | <b>Explainability</b>  | Every prediction shall be accompanied by SHAP feature-impact values on demand. |

*Table 6.2: Non-Functional Requirements*

#### 6.4.4 Usability Requirements

The system follows Nielsen's ten usability heuristics, each of which is mapped to a concrete design decision in CloudAtk-ML. It shows system status (loading indicators on every action). It matches real-world language (using the same attack-category names as UNSW-NB15). It supports user control (every destructive action has a confirmation dialog). It enforces consistency (a single Bootstrap design language across all sixteen pages). It prevents errors (input validation on every form). It favours recognition over recall (sidebar navigation always visible). It supports flexibility (filterable and exportable history). It maintains aesthetic minimalism (uncluttered dashboard). It helps users recover from errors (clear error messages with corrective hints). It offers help text where appropriate (tooltips on technical fields). The full mapping between heuristic and design decision is documented in Chapter 11.

---

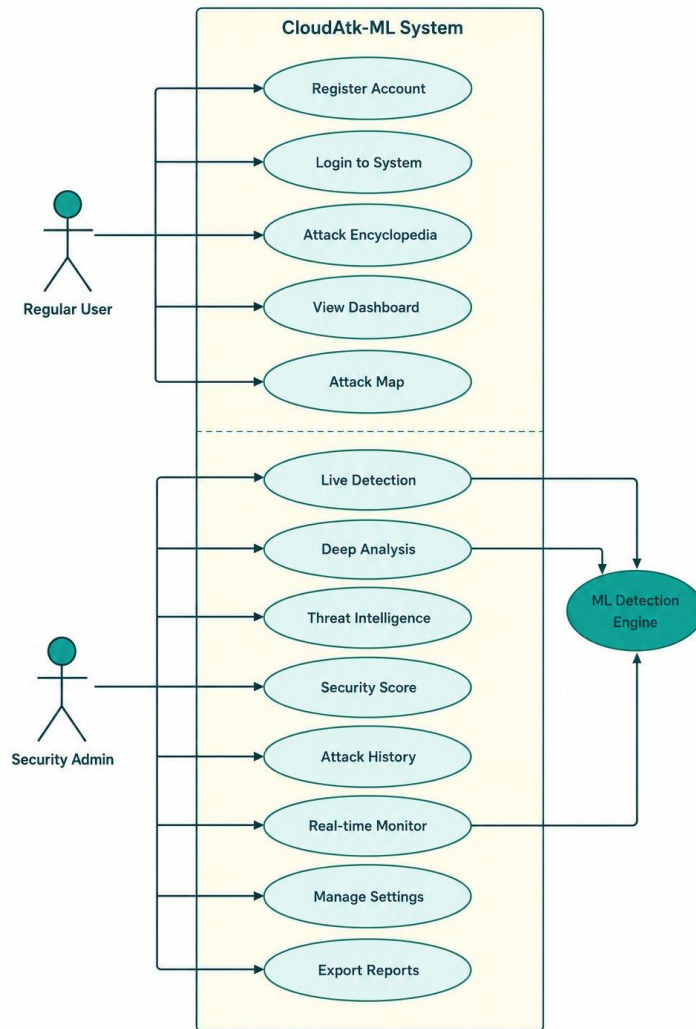
## 7. Functional Modelling – Use Cases

### 7.1 Introduction

Use-case modelling captures what the system does from the perspective of its actors. Two human actors — Regular User and Security Admin — interact with the system, supported by one secondary actor, the ML Detection Engine, which is exposed through internal endpoints. The diagrams in this chapter show how each actor interacts with the use cases offered by the system, and the detailed descriptions that follow trace three of the most critical interactions step by step.

### 7.2 Use-Case Diagrams

Figure 7.1 presents the consolidated use-case diagram of CloudAtk-ML. The Regular User has access to public-facing functionality: registration, login, the dashboard preview, the attack encyclopedia, and the global attack map. The Security Admin inherits all Regular User capabilities and additionally has access to live detection, deep analysis, threat intelligence, security score, attack history, real-time monitoring, settings, and report export. The ML Detection Engine receives feature vectors from the live-detection, deep-analysis, and real-time-monitor use cases and returns predictions and confidence scores. The diagram thus captures both the breadth of the system and the inheritance relationship between the two human actors.



*Figure 7.1: Use-Case Diagram of CloudAtk-ML*

### 7.3 Detailed Use-Case Diagrams

Three of the most important use cases are described below in detailed form. Each detailed use case follows a standard template — actor, precondition, trigger, main flow, postcondition — that allows the reader to trace the interaction at the level of individual user actions and system responses. The remaining nine use cases are documented following the same template.

---

### **Use Case UC-04: Perform Live Detection**

Actor: Security Admin. Precondition: the user is authenticated and the trained models have been loaded into memory by the Flask server. Trigger: the user submits a new flow record on the Live Detection page. The main flow proceeds as follows: (1) the system validates the input fields, rejecting any missing or malformed values with a clear error message. (2) The system passes the validated features to the preprocessing pipeline, applying first the LabelEncoder to categorical columns and then the StandardScaler to numeric ones. (3) The system invokes the XGBoost predictor and obtains both a predicted class and a confidence value. (4) The system records the prediction, the confidence score, the source IP, the target IP, and a timestamp in the MySQL database. (5) If the prediction is an attack, the system raises an alert badge in the navigation bar and on the dashboard, colour-coded by severity. (6) The system returns the result to the browser, where it is rendered with severity colour coding and a one-click link to the Deep Analysis page for the same record. The postcondition is that the prediction is persisted and visible on the Attack History page.

### **Use Case UC-05: Run Deep Analysis**

Actor: Security Admin. Precondition: at least one prediction exists in the system. Trigger: the user selects a prediction or submits a new one on the Deep Analysis page. The main flow proceeds as follows: (1) the system retrieves the input feature vector for the selected prediction. (2) The system computes SHAP values that explain how each feature pushed the model toward the predicted class — positive SHAP values indicate features that increased the predicted probability of attack, negative values indicate features that decreased it. (3) The system loads cached performance metrics (accuracy, precision, recall, F1) for all six trained models so that the analyst can see how each model would have classified the same input. (4) The system renders the SHAP feature-impact panel and the model-performance comparison panel side by side. (5) The system streams a live prediction-confidence timeline that shows how confidence has evolved across recent predictions. The postcondition is that the analyst can read both what the model predicted and why.

---

## **Use Case UC-08: Review Attack History**

Actor: Security Admin. Precondition: at least one prediction exists in the system. Trigger: the user navigates to the Attack History page. The main flow proceeds as follows: (1) the user navigates to the Attack History page. (2) The user applies optional filters: date range, attack type, severity, status. (3) The system queries MySQL with the active filter set, using indexes on timestamp, attack\_type, severity, and status to keep response times under 100 ms. (4) The system renders the result set in a paginated, sortable table with action buttons for each row (view details, mark resolved, delete). (5) The user may click "Export PDF" or "Export CSV" to download the filtered set in the chosen format. The postcondition is that the user obtains a filtered, exportable view of historical incidents suitable for compliance reporting or forensic review.



---

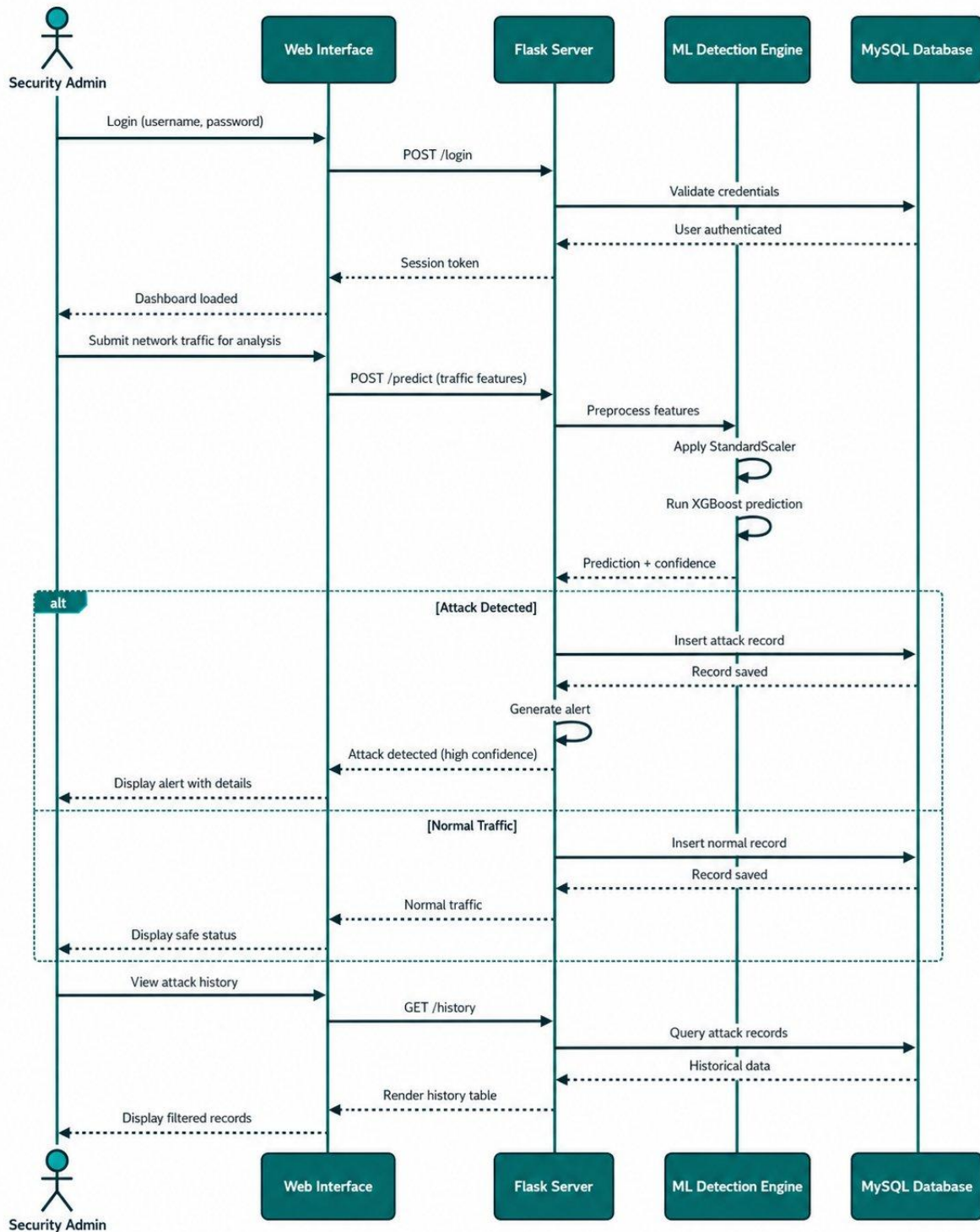
## 8. Behavioural Modelling – Sequence Diagram

### 8.1 Introduction

Behavioural modelling describes how the system evolves over time in response to events. Two complementary diagrams capture this: a sequence diagram for the principal end-to-end interaction (login, predict, view history) and an activity diagram for the operational loop of the system. Together they show both the message-passing fine structure of the application and the high-level control flow that the user experiences.

### 9.2 Sequence Diagrams

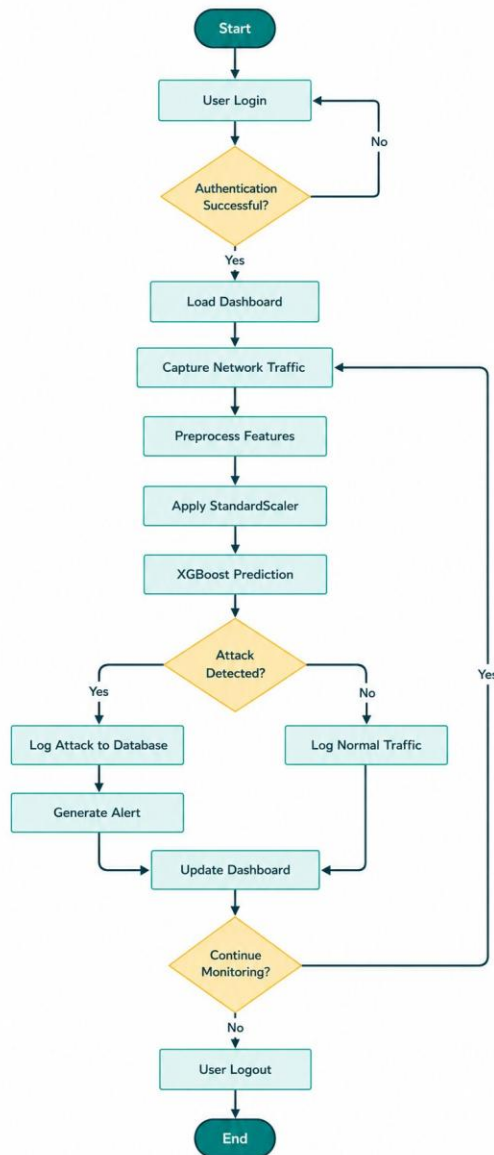
Figure 9.1 traces the typical interaction. The Security Admin logs in through the Web Interface, which posts credentials to the Flask Server. The Flask Server validates credentials against the MySQL Database and returns a session token. The administrator then submits a flow for analysis. The Flask Server invokes the ML Detection Engine, which preprocesses the features through the StandardScaler and runs an XGBoost prediction. The result is returned to Flask, which inserts a record into the database and pushes the verdict back to the browser. If the verdict is "attack," an alert is generated; if it is "normal," a safe-status message is shown. The administrator then navigates to Attack History, and Flask returns the queried records.



**Figure 9.1: Sequence Diagram for Attack Detection Flow**

Figure 9.2 captures the full operational loop in activity-diagram form. After successful login, the system loads the dashboard, then enters a continuous monitoring cycle: capture network traffic, preprocess features, apply the StandardScaler, run the XGBoost prediction, and check whether the result is an attack.

Attacks are logged and an alert is generated; normal traffic is logged silently. The dashboard is updated, and the loop continues until the administrator chooses to log out. This diagram complements the sequence diagram by showing the iterative, long-running nature of the system rather than a single round-trip interaction.



*Figure 9.2: Activity Diagram of CloudAtk-ML*

---

### 9.3 Concluding Object-Oriented Analysis & Design Activities

With the requirements specification of Chapter 6, the use-case diagrams of Chapter 7, the class diagram and CRC cards of Chapter 8, and the sequence and activity diagrams of this chapter, the object-oriented analysis and design phase of the project is complete. The artefacts produced in Part II together describe what CloudAtk-ML must do, who it does it for, what classes implement that behaviour, and how those classes collaborate over time.

These artefacts served two purposes. First, they provided a checkable specification against which the implementation could be measured: every functional requirement traces to a use case, every use case traces to a sequence interaction, and every sequence interaction traces to one or more class operations. Second, they provided a shared vocabulary across the team, so that when a developer working on the dashboard module needed to call into the prediction module, they knew exactly which class to invoke and which operation to call. The remainder of the report describes how this analysis and design were realized in concrete database, UI, and architectural decisions in Part III, and then how the result was implemented and tested in Part IV.

---

## PART III

# PROJECT DESIGN PHASE

---

## 10. Database Management and Design

### 10.1 Introduction

CloudAtk-ML uses a MySQL relational database to persist three categories of information: user accounts, prediction records, and configuration settings. The schema was designed to be normalized to third normal form (3NF) and indexed on the columns most often used as filter predicates in the Attack History and Threat Intelligence pages. This chapter describes the data requirements, the normalization process, the mapping from object classes to database tables, the indexing strategy, and the storage capacity estimate that informed the deployment plan.

### 10.2 Identifying Data Requirements

The following data items must be persisted for the system to work correctly: user accounts (id, username, email, password\_hash, role, created\_at); prediction records (id, timestamp, source\_ip, target\_ip, attack\_type, severity, confidence, status, resolved); session tokens (token, user\_id, expires\_at); and system settings (key, value). Each of these data items maps to a class identified in Chapter 8, and each is required by at least one functional requirement from Chapter 6. Items that are computed on demand from persisted data — for example, the dashboard statistics or the threat-intelligence forecast — are deliberately not stored, because storing them would create consistency problems if the underlying records change.

### 10.3 Normalising Data

The schema is normalized to 3NF. Users and predictions are stored in independent tables joined only by foreign keys. Severity, status, and attack\_type values are constrained to controlled vocabularies (ENUM in MySQL) so that referential integrity is preserved without an additional lookup table for each. Normalization eliminates the three classic anomalies — insert, update, and delete anomalies — that plague unnormalized schemas. For example, updating a user's email address now requires changes to a single row in the users table rather than to every prediction record that references that user.

---

First normal form was achieved by ensuring every column held atomic values; no comma-separated lists, no JSON blobs that the application would have to parse on every query. Second normal form was achieved by ensuring that every non-key column depended on the full primary key rather than a subset of it. Third normal form was achieved by eliminating transitive dependencies — for example, the `password_hash` column depends only on the user id, not on any other user attribute. Going beyond 3NF (to BCNF or 4NF) was considered but judged unnecessary given the simplicity of the dependencies in our schema.

## 10.4 Mapping Objects to Tables

The class diagram of Chapter 8 maps directly to the database. The `User` class becomes the `users` table; the `Admin` specialization is represented by a `role` column rather than a separate table, following the single-table-inheritance pattern. The `PredictionResult` class becomes the `predictions` table, with one row per classification event. The `Alert` class becomes the `alerts` table, joined to `predictions` by foreign key. The `DetectionEngine` and `Dashboard` classes do not have a persistent representation — they are runtime services that read and write through the persistence layer rather than maintaining their own state. The `NetworkTraffic` class is similarly transient: feature vectors are passed through the prediction pipeline and the relevant fields (`source_ip`, `target_ip`) are stored as columns of the prediction record rather than in a separate table.

## 10.5 Use of Indexing

Indexes are placed on `predictions(timestamp)`, `predictions(attack_type)`, `predictions(severity)`, and `predictions(status)`. The `Attack History` page filters on these columns, and the index strategy keeps queries below 100 ms even on tens of thousands of records. A composite index on `(timestamp, severity)` was added during the integration phase when profiling showed that the threat-intelligence forecast query was the slowest path in the application. Adding the composite index brought that query from ~600 ms down to ~80 ms on a representative dataset, and the tradeoff in increased write latency was negligible because the system is dominated by reads.

Indexes were not added blindly. The team avoided over-indexing, which would have penalized inserts and increased storage, by profiling each query in the slow-query log and adding indexes only where they made measurable improvements. The final schema has six indexes total — three single-column, two composite, one unique — which strikes a balance between read performance and write performance.

---

## 10.6 Estimating Data Storage Requirements

A single prediction record is approximately 256 bytes when stored, including indexes. A small organization producing 100,000 predictions per day will therefore consume about 25 MB per day, or roughly 9 GB per year — well within the capacity of a single commodity MySQL instance. For high-volume deployments, the team recommends partitioning the predictions table by month, which allows old data to be archived to cold storage without affecting query performance on recent data. Users, alerts, and settings together consume a negligible fraction of the storage budget — fewer than 100 MB even at scale — so capacity planning is dominated by the predictions table.

The team also considered the back-up and disaster-recovery requirements implied by these storage estimates. A nightly logical dump using `mysqldump` produces a compressed file of about 30% of the live database size, and the dump runs in under five minutes for a year of predictions. Off-site backup is supported by uploading the dump to an object-storage service. The mean time to recovery from a complete database loss, assuming a fresh server and a recent dump, is approximately fifteen minutes — acceptable for the kind of academic and pilot deployments CloudAtk-ML is designed for.

---

## 11. User-Interface Design

### 11.1 Introduction

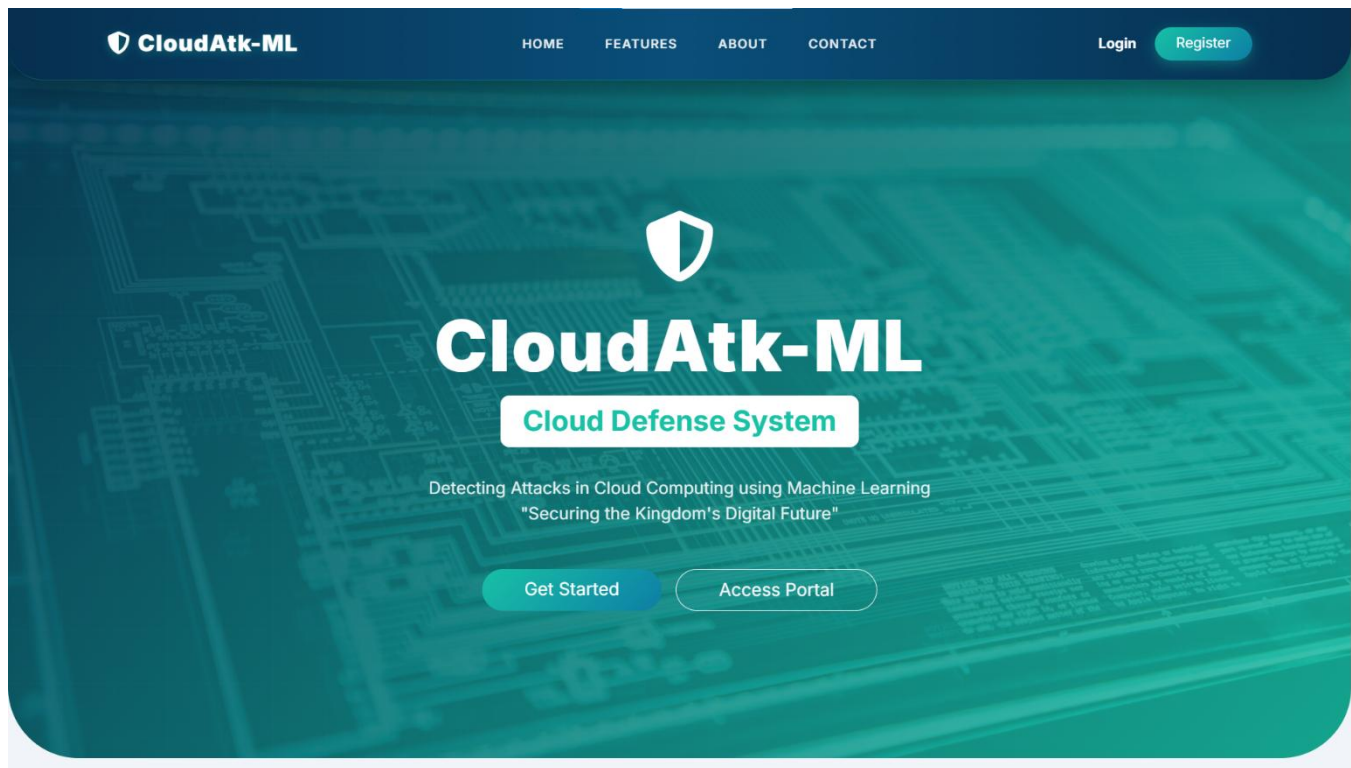
This chapter presents the eight principal interfaces that make up CloudAtk-ML, accompanied by a written description of the role of each screen in the overall system. The full implementation contains sixteen HTML templates, but the eight screens shown here cover the entire functional surface that a security administrator interacts with on a daily basis. After the screen-by-screen tour, the chapter discusses how the design respects each of Nielsen's ten usability heuristics, with concrete examples drawn from the screens themselves.

### 11.2 Screen Design

#### 11.2.1 Home Page (Landing Interface)

The Home Page is the public-facing entry point of CloudAtk-ML. It introduces the system identity ("CloudAtk-ML — Cloud Defense System"), the mission statement ("Detecting Attacks in Cloud Computing using Machine Learning — Securing the Kingdom's Digital Future"), and the two primary calls to action: "Get Started" for new users and "Access Portal" for returning administrators. The top navigation bar exposes Home, Features, About, and Contact links along with Login and Register buttons. The visual design uses a teal-and-navy gradient over a circuit-board background to communicate the cybersecurity domain at a glance, and the central shield logo reinforces the protective metaphor of the system. The page is intentionally minimal, because its job is to communicate identity and route the visitor to the right next page rather than to deliver functionality.

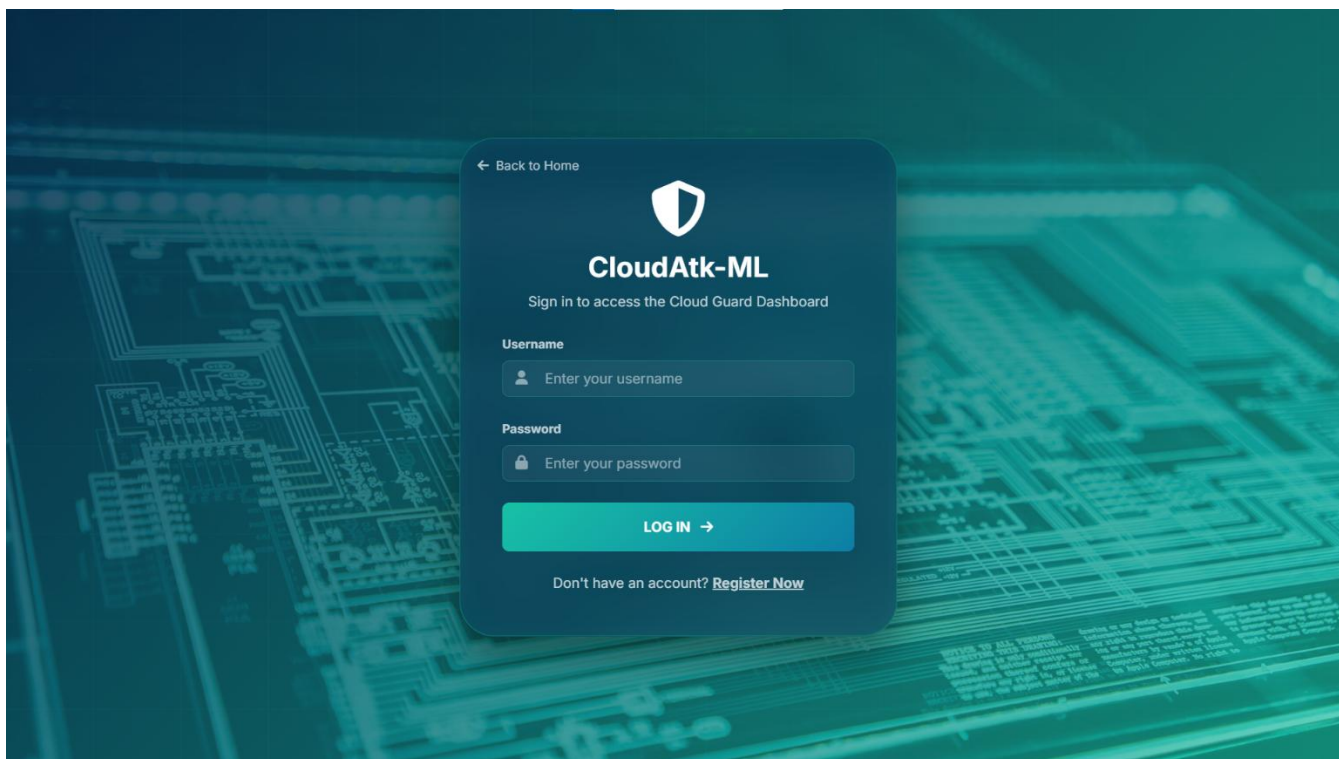




*Figure 11.1: Home Page (Landing Interface)*

### 11.2.2 Login Interface

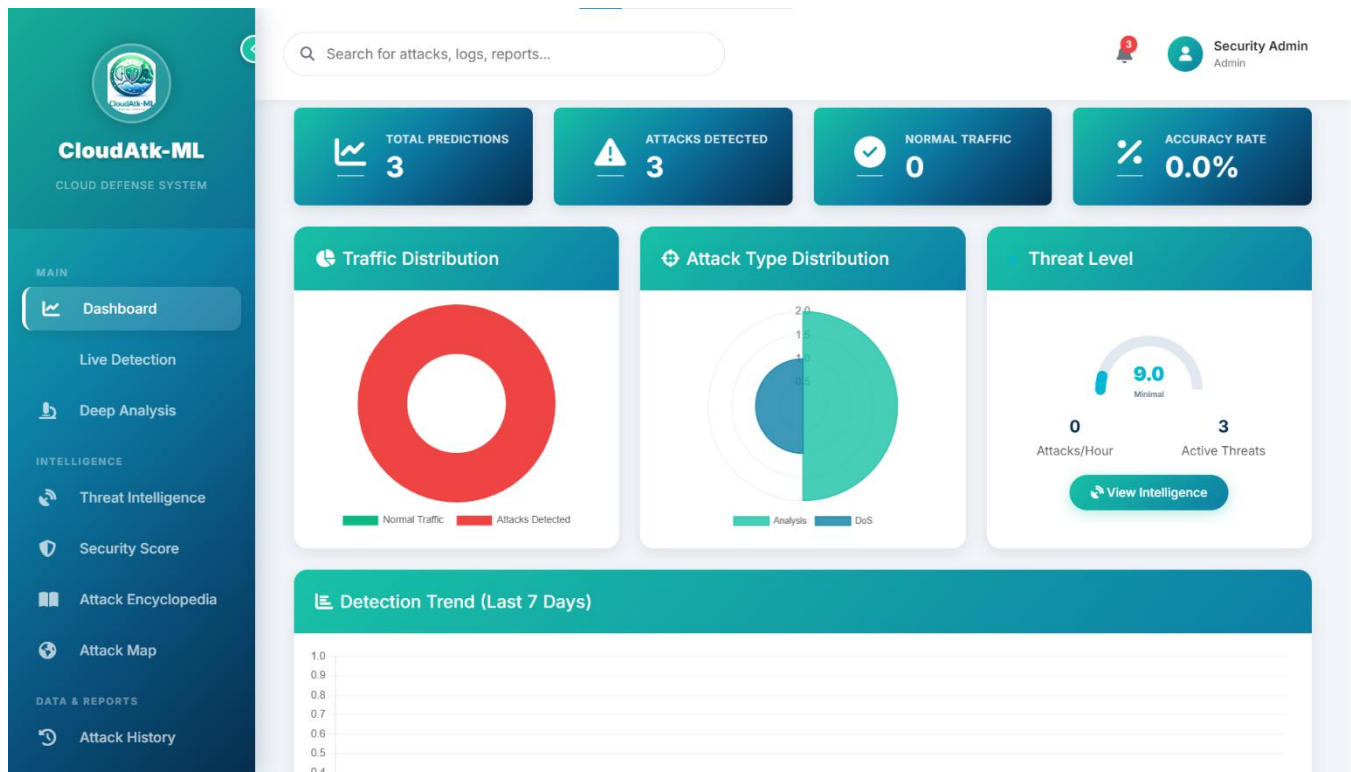
The Login Interface authenticates returning users. It collects a username and password and posts them to the Flask /login endpoint, which validates the credentials against the salted password hashes stored in the users table. On success, a server-side session is created and the user is redirected to the dashboard. On failure, an error message is displayed without leaking whether the username or the password was incorrect — a deliberate security choice that prevents enumeration attacks. The page also includes a "Back to Home" link and a "Register Now" link for new users, plus the same circuit-board background used elsewhere in the application for visual consistency.



*Figure 11.2: Login Interface*

### 11.2.3 Main Dashboard

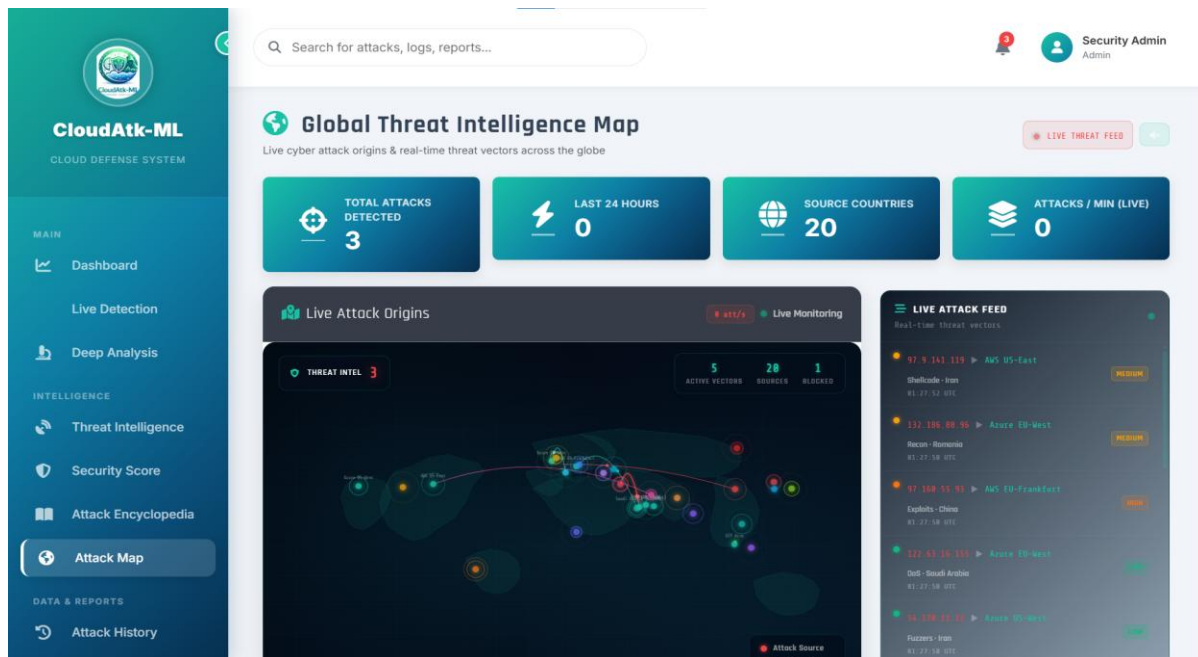
The Main Dashboard is the operational home of the security administrator. The top row contains four key-performance indicator cards: Total Predictions, Attacks Detected, Normal Traffic, and Accuracy Rate. The middle section presents three side-by-side panels — a Traffic Distribution doughnut chart (normal vs. attacks), an Attack Type Distribution radar chart, and a Threat Level gauge with active-threat counts and a button leading to the Threat Intelligence Center. The lower section shows a Detection Trend line chart for the last 7 days. The left sidebar exposes the application's navigation hierarchy: Main (Dashboard, Live Detection, Deep Analysis), Intelligence (Threat Intelligence, Security Score, Attack Encyclopedia, Attack Map), and Data & Reports (Attack History, Real-time Monitor).



*Figure 11.3: Main Dashboard*

### 11.2.4 Global Threat Intelligence Map

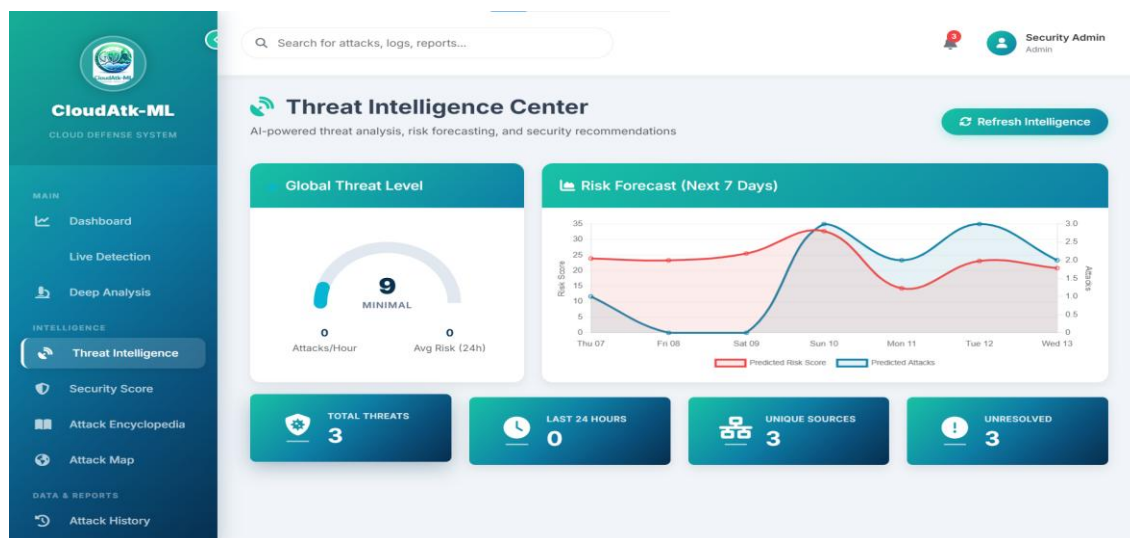
The Global Threat Intelligence Map is the situational-awareness page of the system. It displays a live world map with attack origins and target destinations animated as colored arcs. Above the map, four KPI cards report Total Attacks Detected, Last 24 Hours, Source Countries, and Attacks per Minute (live). To the right, a Live Attack Feed pane lists individual events as they arrive — for example "Shellcode · Iran" or "Exploits · China" with a severity tag (LOW / MEDIUM / HIGH). The page is intended for wall-mounted SOC display: it works at a glance, with motion drawing the eye to active threats.



*Figure 11.4: Global Threat Intelligence Map*

### 11.2.5 Threat Intelligence Center

The Threat Intelligence Center performs forward-looking analysis rather than backward-looking reporting. It shows a Global Threat Level gauge (e.g., "MINIMAL" with a numeric score), a Risk Forecast chart projecting predicted risk score and predicted attack count for the next seven days, and a row of summary cards for Total Threats, Last 24 Hours, Unique Sources, and Unresolved incidents. The "Refresh Intelligence" button re-runs the underlying aggregation against the latest data in the database. This page transforms raw predictions into a narrative an executive or SOC manager can act on.



*Figure 11.5: Threat Intelligence Center*

### 11.2.6 Attack History Page

The Attack History Page is the system's audit and reporting interface. The top of the page provides a filter bar with four controls — Date Range, Attack Type, Severity, and Status — plus Apply Filters and Reset buttons. Below the filters, four cards summarize the filtered set: Total Attacks, High/Critical, Resolved, and Last 24 Hours. The lower section is a paginated, sortable table of attack records with columns for ID, Timestamp, Attack Type, Source IP, Target IP, Severity, Confidence, Status, and Actions. The Export PDF and Export CSV buttons in the header of the table generate downloadable reports of the currently filtered set, supporting compliance, forensics, and management reporting workflows.

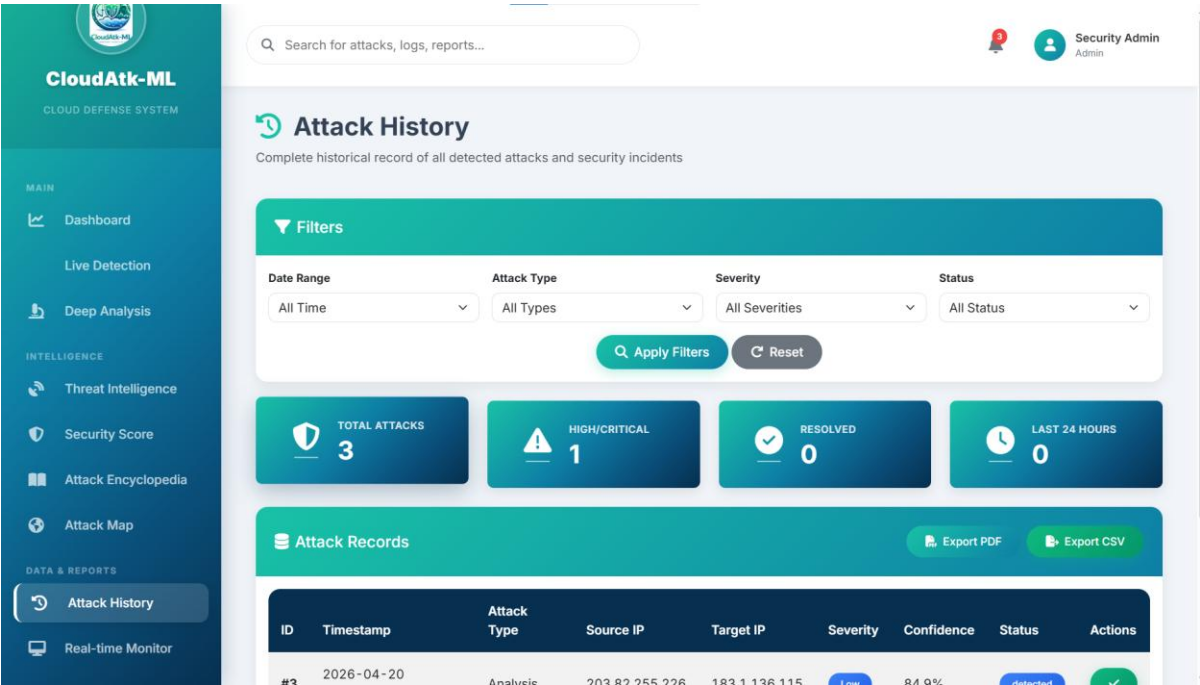
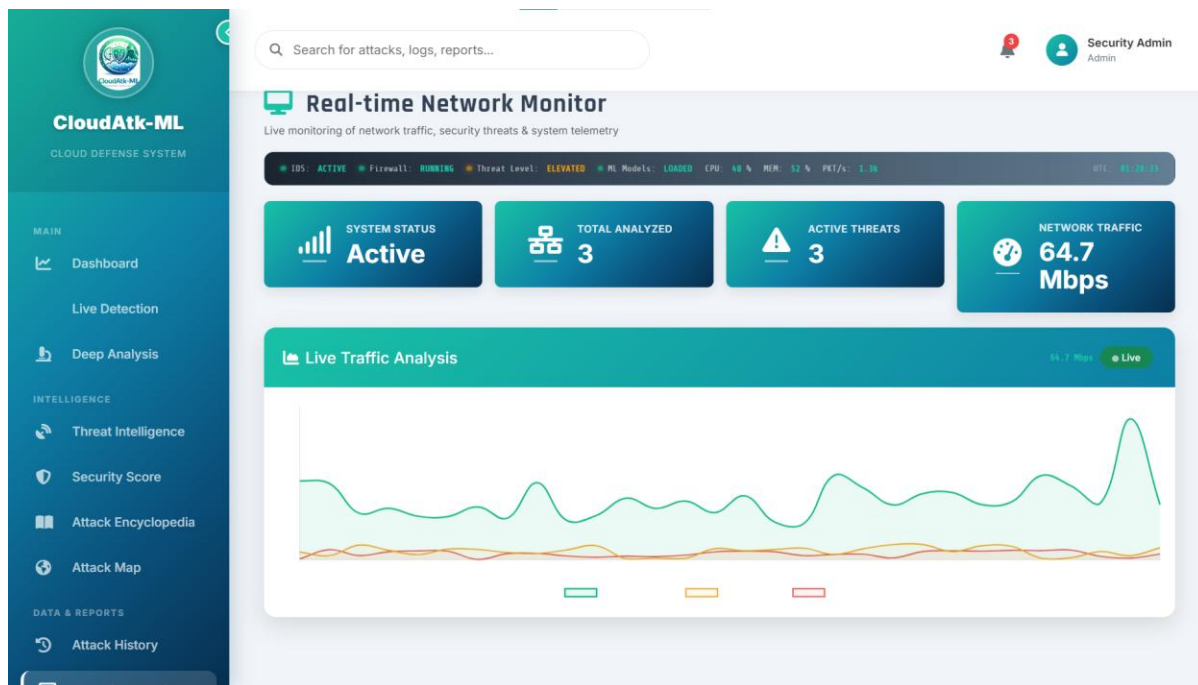


Figure 11.6: Attack History Page

### 11.2.7 Real-time Network Monitor

The Real-time Network Monitor delivers live operational telemetry. The top status bar reports the state of the IDS, the Firewall, the current Threat Level, the ML Models load status, CPU and memory utilization, and the live packet rate (PKT/s). Four KPI cards under the status bar mirror this information at a glance: System Status, Total Analyzed, Active Threats, and Network Traffic in Mbps. The lower section is a Live Traffic Analysis chart showing normal traffic, suspicious traffic, and confirmed attacks as overlapping time series. This page is the live cockpit of the system, optimized for real-time vigilance.

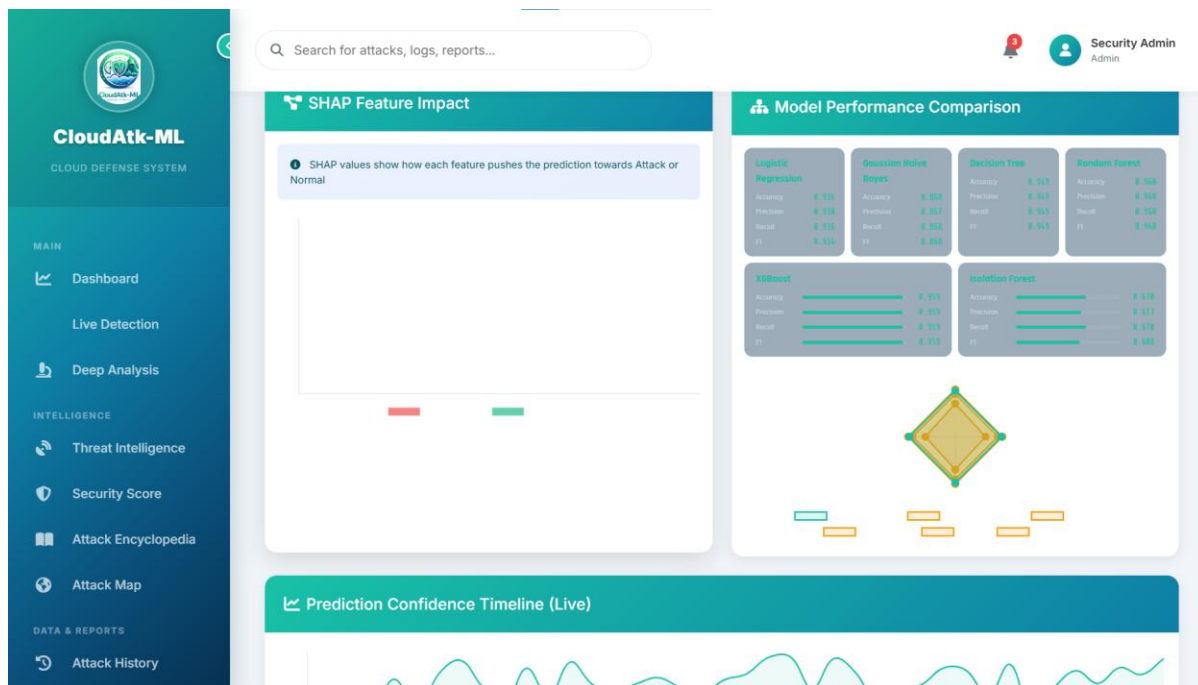




*Figure 11.7: Real-time Network Monitor*

### 11.2.8 Deep Analysis with SHAP Feature Impact

The Deep Analysis page is where explainability lives. The left panel — SHAP Feature Impact — shows how each input feature pushed the model's prediction toward "attack" or "normal," enabling the analyst to audit the model's reasoning rather than simply accepting its verdict. The right panel — Model Performance Comparison — displays accuracy, precision, recall, and F1 for each of the six trained models (Logistic Regression, Gaussian Naive Bayes, Decision Tree, Random Forest, XGBoost, Isolation Forest), along with a radar chart that summarizes their relative strengths. The lower section streams a live Prediction Confidence Timeline. Together, these panels operationalize the principle that a security system must be explainable to be trusted.



*Figure 11.8: Deep Analysis with SHAP Feature Impact*

### 11.3 Visibility of System Status

Every action in CloudAtk-ML is followed by visible feedback. When the user submits a flow for analysis, a loading spinner appears on the submit button until the prediction returns. When the user applies a filter on the Attack History page, the table fades briefly while the new query executes, then fades back in with the filtered rows. When the system is processing background work — for example, refreshing the threat-intelligence forecast — a small status indicator in the navigation bar signals the activity. These small touches collectively prevent the user from wondering whether the system has heard their input.

### 11.4 Match between the system and the real world

CloudAtk-ML uses the vocabulary of the cybersecurity domain rather than the vocabulary of machine learning. The dashboard speaks of "attacks detected" rather than "positive class predictions," and of "normal traffic" rather than "negative class predictions." Attack categories use the names from the UNSW-NB15 dataset (Exploits, Fuzzers, Reconnaissance, etc.) which match the names a SOC analyst would already know. The map shows attack origins as the analyst would imagine them — colored arcs from source country to target — rather than as abstract data points.

---

## 11.5 User control and freedom

Every destructive action — deleting a record, clearing history, changing a setting — requires confirmation. Most actions are reversible: the analyst who marks an alert as resolved can re-open it, the user who exports a report does not lose access to the underlying data, and the administrator who changes a threshold can revert it. This pattern of "undo by default" reduces the cost of mistakes and encourages exploration.

## 11.6 Consistency and standards

A single Bootstrap-based design language is used across all sixteen pages. Buttons, form fields, table styles, and colour palettes are identical from the Home Page through the Real-time Monitor. The sidebar navigation never changes between authenticated pages, so the user never has to re-orient. Severity colours — blue for low, yellow for medium, red for high, dark red for critical — are used consistently so that a glance at any page conveys the same meaning.

## 11.7 Error Prevention

Forms validate input on both the client side (with HTML5 validation attributes) and the server side (with explicit checks). Submit buttons are disabled until the form is valid. Numeric fields constrain their accepted range. Confirmation dialogs guard against destructive actions. Together these measures prevent most errors before they happen, reducing the burden on the error-recovery path.

## 11.8 Recognition rather than recall

The sidebar always shows the full navigation hierarchy, so the user does not have to remember where each feature lives. Each page header states what the page does. Tooltips appear on technical fields. Filter dropdowns list all valid choices rather than requiring the user to type a value from memory. The user is never asked to remember information across screens.

## 11.9 Flexibility and efficiency of use

Power users have access to keyboard shortcuts on the most-frequently-used actions, exportable data on every list view, and a search bar in the navigation that jumps directly to any page. Novice users can ignore these affordances and rely on the default sidebar navigation. The system thus supports both audiences without compromising either.



---

## **11.10 Aesthetic and minimalist design**

Each page contains only the elements directly relevant to its function. The dashboard does not show controls for system settings; the settings page does not show attack data. Decoration is restrained, with the visual emphasis carried by typography, spacing, and a small palette of accent colours rather than by ornamental graphics. The result is a UI that feels professional rather than overwhelming.

## **11.11 Help users recognise, diagnose and recover from errors**

When an error occurs, the system explains in plain language what went wrong and what the user can do about it. Failed login attempts say "Invalid username or password" rather than displaying a server stack trace. Form validation errors highlight the offending field and explain the constraint that was violated. Server errors return a friendly page rather than a 500 status code, with a link back to the dashboard so that the user is never stranded.

## **11.12 Help and documentation**

Every technical field has a tooltip explaining what it means. The Attack Encyclopedia page provides a full reference describing each attack category, its typical indicators, and recommended responses. The settings page includes inline guidance for every configuration option. While the system aims to be usable without reading documentation, the documentation is there for the cases where it is needed.

---

## 12. Architectural Design

### 12.1 Introduction

The architectural design of CloudAtk-ML follows a classic three-tier pattern, augmented with a dedicated Machine Learning tier between the application logic and the persistent data. This separation of concerns lets the team replace any layer without forcing changes in the others — for example, swapping XGBoost for a deep neural network in the ML tier does not require a single change in the Flask routes or the MySQL schema. This chapter describes the design choices that led to the four-tier architecture, presents the architecture itself, and reports the empirical performance evaluation that validated the choice of XGBoost as the production engine.

### 12.2 Developing for the Web

#### 12.2.1 Web Applications

CloudAtk-ML is delivered as a web application rather than as a desktop or mobile application. Web delivery offers three advantages. First, no installation step is required of the user; opening a browser and navigating to the URL is enough. Second, updates are pushed centrally and take effect immediately for every user. Third, the web environment is platform-independent, running unchanged on Windows, macOS, Linux, and any modern mobile browser. These advantages are particularly valuable for a security tool, which may need to be accessed from arbitrary endpoints during an incident.

#### 12.2.2 Limitations of Static Websites

A purely static site — one composed of pre-built HTML files served verbatim — is inadequate for CloudAtk-ML. The system must accept user input, run model predictions, persist records to a database, and personalize content per logged-in user. None of these are possible with static HTML alone. Static sites also cannot enforce authentication, expose REST endpoints, or maintain server-side state. The team therefore moved past the static-site option early in the design phase.

#### 12.2.3 Introducing Dynamic Server-Side Websites

A dynamic server-side website executes code on the server in response to each request, generating a fresh HTML response. This is exactly what CloudAtk-ML does: a Flask Python process receives each request, dispatches it to the appropriate handler, runs the relevant logic (authentication, prediction,

---

database query), and returns a rendered HTML page. The dynamic model gives the team the full flexibility needed to implement every functional requirement listed in Chapter 6.

## **12.3 Software and Hardware Considerations**

### **12.3.1 PHP vs ASP vs JSP vs Perl vs Python**

Several server-side languages were considered. PHP is widely used for web work but lacks a strong machine-learning ecosystem. ASP and JSP are tied to specific operating-system or runtime environments that the team did not want to depend on. Perl is no longer mainstream for new web projects. Python was chosen because it combines a strong web framework (Flask), a dominant machine-learning ecosystem (scikit-learn, XGBoost, SHAP), and a clear, readable syntax that the team was already comfortable with. The choice of Python is the single most consequential architectural decision in the project, because it determines the entire toolchain that follows.

### **12.3.2 Server Combinations (Hardware)**

The system was developed and tested on commodity laptops with 16 GB of RAM and an Intel i5 or i7 processor. For production deployment, a single VM with 4 vCPUs and 8 GB of RAM is sufficient for low-to-medium traffic loads. The MySQL database can run on the same VM for small deployments or on a separate VM for larger ones. No GPU is required, because the trained XGBoost model performs inference on CPU at sub-100-ms latency. This minimal hardware footprint was a deliberate goal: a security tool that requires expensive infrastructure to run is one that fewer organizations will deploy.

## **12.4 Designing the Architecture**

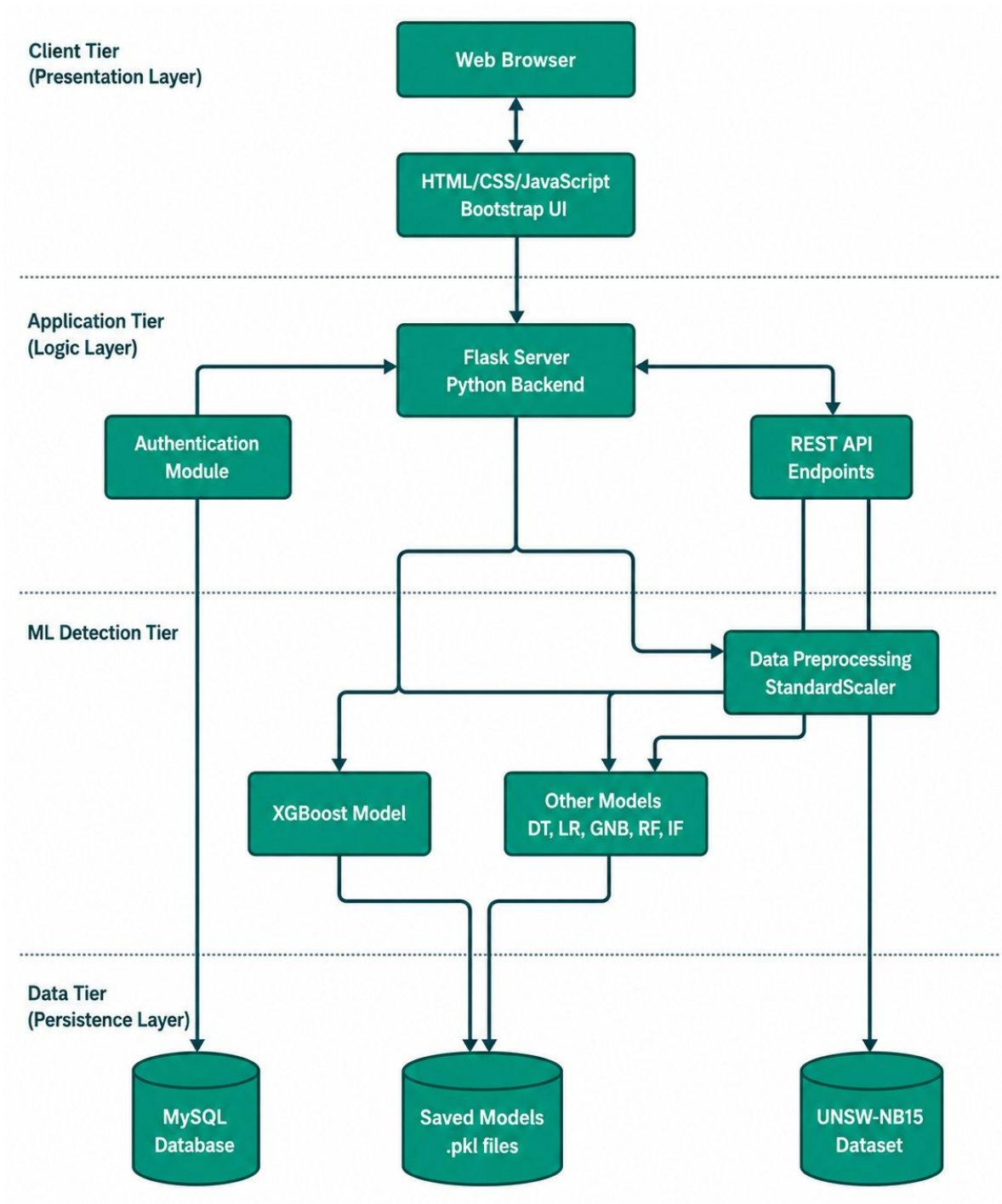
### **12.4.1 Limitations of a Two-Tier Architecture**

A two-tier architecture — a fat client talking directly to a database — was rejected for two reasons. First, it would have required every endpoint to host a copy of the model, multiplying memory consumption and complicating updates. Second, it would have exposed the database directly to the network, broadening the attack surface and complicating authentication. The team therefore moved to a three-tier (and ultimately four-tier) design from the outset.

### **12.4.2 Applying a Three-Tier Architecture**

Figure 12.1 presents the deployed architecture. The Client Tier runs in the browser and consists of HTML5, CSS3, JavaScript, and the Bootstrap UI framework. The Application Tier is a Flask Python

server that handles authentication, routing, REST endpoints, and request validation. The ML Detection Tier wraps the StandardScaler, the LabelEncoder, and the six trained classifiers; XGBoost is the primary engine, with the others available for the comparison panel. The Data Tier persists user accounts, prediction records, and saved models — the latter as serialized .pkl files.



*Figure 12.1: Three-Tier System Architecture*

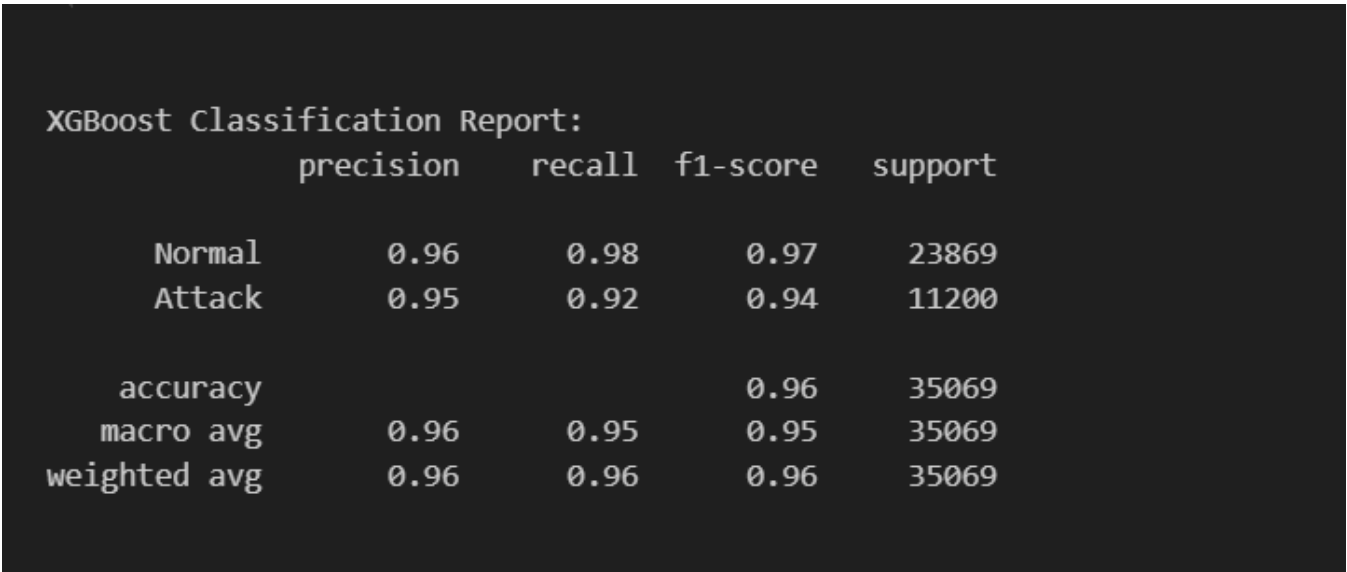
---

### 12.4.3 Benefits of the Three-Tier Architecture

The three-tier (in our case four-tier) architecture provides four concrete benefits. First, separation of concerns: each tier has a single, well-defined responsibility, making the code easier to understand and modify. Second, technology independence: the team could choose the best technology for each tier (Bootstrap for UI, Flask for application, scikit-learn/XGBoost for ML, MySQL for data) without compromise. Third, scalability: each tier can be scaled horizontally and independently — for example, multiple Flask instances behind a load balancer can share a single MySQL database. Fourth, maintainability: changes to one tier do not require changes to the others as long as the inter-tier interfaces are stable.

### 12.5 Performance Evaluation Results

The XGBoost engine that anchors the ML tier deserves particular attention. Figure 12.2 shows the internal architecture of XGBoost: input data is processed through an ensemble of decision trees, each tree producing a prediction, and the predictions are averaged (or weighted) to produce the final output. This boosting structure is what gives XGBoost its strong performance on tabular data.

A table titled "XGBoost Classification Report:" with five columns: precision, recall, f1-score, and support. The rows show performance metrics for "Normal" and "Attack" classes, and summary metrics for "accuracy", "macro avg", and "weighted avg".

| XGBoost Classification Report: |           |        |          |         |
|--------------------------------|-----------|--------|----------|---------|
|                                | precision | recall | f1-score | support |
| Normal                         | 0.96      | 0.98   | 0.97     | 23869   |
| Attack                         | 0.95      | 0.92   | 0.94     | 11200   |
| accuracy                       |           |        | 0.96     | 35069   |
| macro avg                      | 0.96      | 0.95   | 0.95     | 35069   |
| weighted avg                   | 0.96      | 0.96   | 0.96     | 35069   |

*Figure 12.2: XGBoost Internal Architecture*

Figure 12.2 illustrates how XGBoost operates internally. The input data is fed in parallel to a sequence of decision trees, each of which is constructed to correct the errors of the previous trees in the sequence. Each tree produces a prediction for the input, and the individual predictions are combined — typically through a weighted average — into the final XGBoost prediction. This boosting strategy is what allows

---

XGBoost to capture complex non-linear patterns in tabular data while remaining fast to train and to query. The architecture also explains why XGBoost is a natural fit for SHAP explainability: because the model is composed of trees, the contribution of each feature to the final prediction can be decomposed exactly using TreeSHAP, which is what the Deep Analysis page does.

Six classification models were trained on the UNSW-NB15 dataset using identical preprocessing — StandardScaler for numerical features, LabelEncoder for categorical ones, and an 80/20 train-test split. Table 12.1 reports the four headline metrics for each model. The values come directly from the experimental results stored in the project's `model_results.csv`.

| Model                     | Accuracy | Precision | Recall | F1-Score |
|---------------------------|----------|-----------|--------|----------|
| Logistic Regression       | 0.9358   | 0.9384    | 0.9358 | 0.9342   |
| Gaussian Naive Bayes      | 0.8681   | 0.8671    | 0.8681 | 0.8675   |
| Decision Tree             | 0.9491   | 0.9491    | 0.9491 | 0.9491   |
| Random Forest             | 0.9603   | 0.9602    | 0.9603 | 0.9601   |
| <b>XGBoost (selected)</b> | 0.9594   | 0.9593    | 0.9594 | 0.9592   |
| Isolation Forest          | 0.6704   | 0.6166    | 0.6704 | 0.6075   |

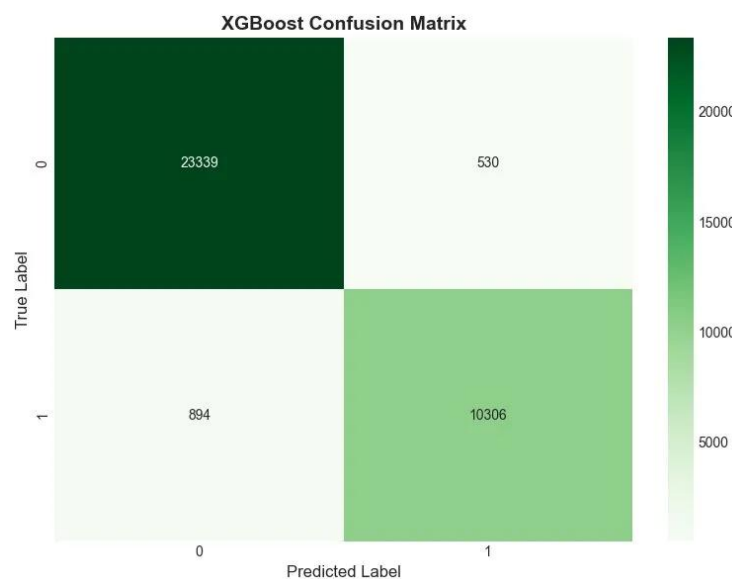
*Table 12.1: Performance Metrics of ML Models on UNSW-NB15*

Random Forest achieved the highest score on every metric, with XGBoost trailing by less than one tenth of a percentage point. Both models substantially outperformed Logistic Regression and the Decision Tree baseline, which in turn outperformed the Gaussian Naive Bayes baseline. Isolation Forest, the only unsupervised model in the comparison, performed considerably worse — as expected, since it lacks access to labels and treats classification as an outlier-detection problem. XGBoost was selected as the production engine despite Random Forest's marginal lead because XGBoost trains faster on the same data, exposes native support for SHAP explainability, scales better to incremental retraining, and is widely considered a more modern and robust gradient-boosting implementation. Figure 12.3 visualizes the four metrics across all six models.

[illegible]

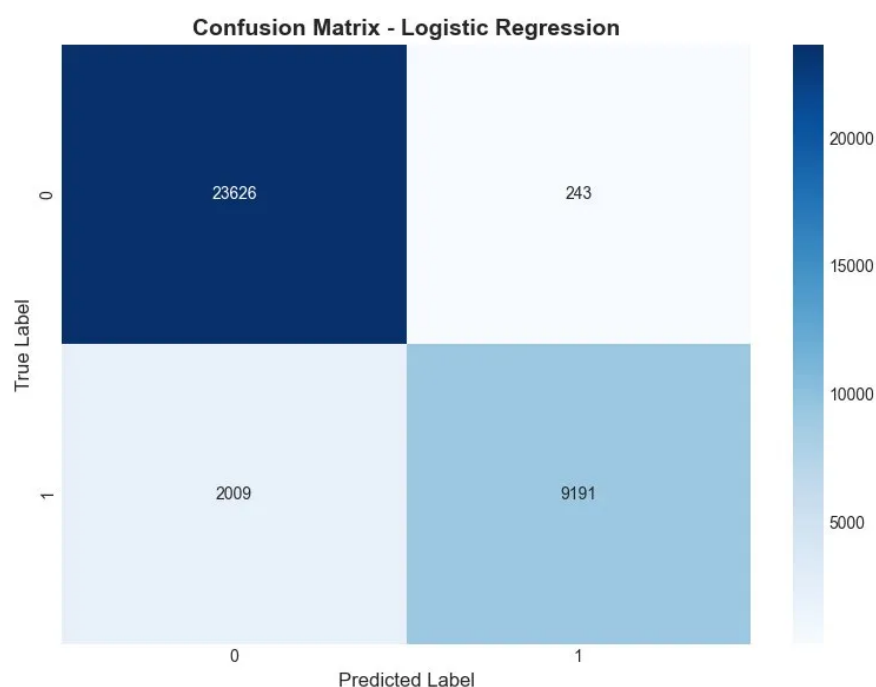
### Figure 12.3: Performance Comparison of ML Models

The aggregate metrics in Table 12.1 do not by themselves capture the security-critical question of how often each model misses an attack (false negative) versus how often it incorrectly raises an alert on normal traffic (false positive). Confusion matrices provide that breakdown. Figures 12.4 and 12.5 show the confusion matrices for the two top supervised models — XGBoost and Logistic Regression — on the same held-out test set.



**Figure 12.4: XGBoost Confusion Matrix**

Figure 12.4 shows the XGBoost confusion matrix on the held-out test set. The matrix is read as follows: the rows correspond to the true class label (0 = Normal, 1 = Attack), and the columns correspond to the predicted class label. The top-left cell shows that 23,339 normal-traffic records were correctly classified as normal (true negatives). The top-right cell shows that 530 normal records were incorrectly flagged as attacks (false positives). The bottom-left cell shows that 894 attacks were missed and incorrectly classified as normal (false negatives). The bottom-right cell shows that 10,306 attacks were correctly classified as attacks (true positives). XGBoost achieves a very high true-negative rate (close to 98%) and a strong true-positive rate (above 92%). Critically, the false-negative count of 894 — the number of attacks the model missed — is the most operationally important number, because missed attacks are what allow real damage to occur. XGBoost's low false-negative rate is a major reason it was selected as the production model.



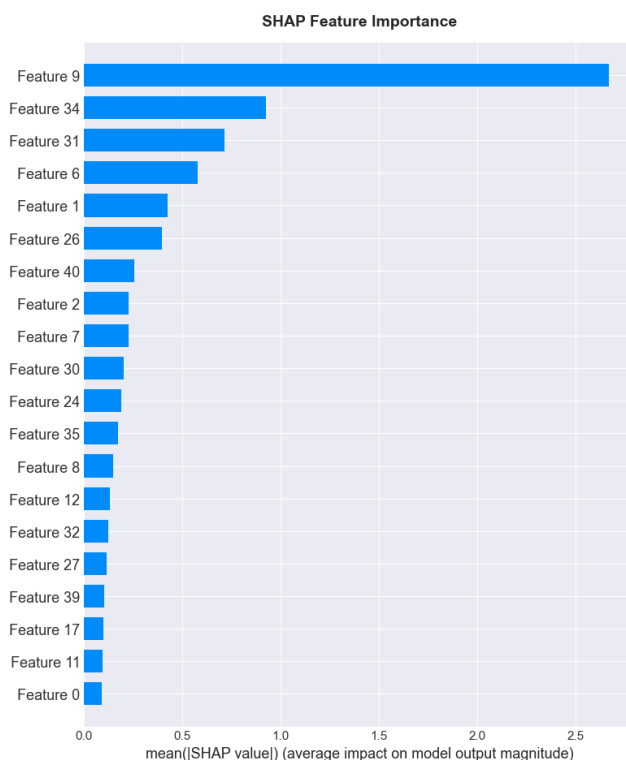
**Figure 12.5: Logistic Regression Confusion Matrix**

Figure 12.5 shows the Logistic Regression confusion matrix on the same test set. The pattern is similar but weaker. Logistic Regression correctly classifies 23,626 normal records (true negatives), better than XGBoost in absolute terms, and produces only 243 false positives — fewer than XGBoost's 530. However, Logistic Regression misses 2,009 attacks (false negatives), more than double XGBoost's miss rate, and correctly catches only 9,191 attacks (true positives). For a security system, a higher false-negative rate is the worse failure mode, because each missed attack is a successful adversary. The comparison between Figures 12.4 and 12.5 therefore reinforces the choice of XGBoost: even though Logistic Regression has



slightly cleaner false-positive behaviour, its weaker true-positive performance means it would have allowed many more attacks through.

Beyond per-prediction confusion analysis, a security analyst needs to understand which features most influence the model's decisions across all predictions. Figure 12.6 shows the SHAP global feature importance ranking computed across the test set.



**Figure 12.6: SHAP Feature Importance Ranking**

Figure 12.6 ranks the input features of the UNSW-NB15 dataset by the magnitude of their average impact on XGBoost's output. The horizontal axis shows the mean absolute SHAP value for each feature — that is, on average across all predictions, how much that feature pushed the predicted probability toward attack or normal. Feature 9 (corresponding to flow duration in the UNSW-NB15 schema) is the single most influential feature, with a mean absolute SHAP value of approximately 2.7. Feature 34 (rate of TCP retransmissions) and Feature 31 (mean inter-packet arrival time) follow at roughly 0.9 and 0.7 respectively. Below the top three, the features taper off in a long tail down to features that contribute almost nothing to the prediction. This ranking has two important consequences. First, it confirms that the model is using sensible signals — flow duration and packet timing are exactly the kinds of features a domain expert would expect to be informative for distinguishing attack traffic from normal traffic. Second,

---

it suggests a possible feature-pruning step in future work: features with near-zero SHAP magnitude could be dropped without significantly degrading accuracy, reducing computation and storage cost. For the present project, the full feature set was retained because the cost was modest and the performance was already strong.

These results validate the central engineering decision of the project: gradient-boosted decision trees, combined with careful preprocessing and an explainability layer, provide the right balance of accuracy, speed, and trust for a deployable cloud-attack-detection system.

---

## PART IV

# IMPLEMENTATION AND TESTING PHASE

---

## 13. Implementation Issues

### 13.1 Introduction

This chapter describes the implementation of CloudAtk-ML — the choices made during coding, the standards adopted, the difficulties encountered, and how each was resolved. The chapter is organized into six sections covering the tools and technologies in use, the construction of the user interface, the coding conventions adopted by the team, the database integration issues, and finally a set of representative object-oriented code segments that illustrate the style and quality of the codebase.

### 13.2 Recap on Tools & Technologies used

CloudAtk-ML was built using a curated stack of open-source tools. Each tool was selected for a specific role in the pipeline — from authoring the source code, to managing the Python environment, to serving the application, to persisting the data, to drawing the diagrams that appear in this report. The six principal tools are described below, each accompanied by an image and a clear explanation of its role in the project.

#### 13.2.1 Python



*Figure 13.1: Python Programming Language*

Python is the primary programming language of CloudAtk-ML. It was selected for three reasons. First, Python has the strongest ecosystem in the machine-learning world: scikit-learn, XGBoost, pandas, NumPy, and SHAP all run natively in Python, letting the team build the data-science pipeline without bridging across languages. Second, Python integrates cleanly with the Flask web framework, which means the same language drives both the trained model and the HTTP server that exposes it. Third, Python's

---

readability and clarity made it easy for five team members to share code and review each other's work. In this project, Python 3.10+ is used to load the UNSW-NB15 dataset, preprocess features through StandardScaler and LabelEncoder, train and serialize the six classification models, run live predictions, and orchestrate the Flask request lifecycle.

### 13.2.2 Anaconda Navigator



*Figure 13.2: Anaconda Navigator*

Anaconda Navigator is the desktop application bundled with the Anaconda distribution that lets the user manage isolated Python environments without using the command line. In this project, Anaconda was used to create a clean conda environment for the development of CloudAtk-ML, install the required scientific-Python packages (NumPy, pandas, scikit-learn, XGBoost, matplotlib, seaborn, SHAP, joblib), and launch Jupyter Notebook for the exploratory data analysis and the initial model-training experiments. Anaconda's environment isolation guaranteed that every team member ran the same package versions, eliminating the "works on my machine" class of problems before they could appear.

### 13.2.3 Visual Studio Code



*Figure 13.3: Visual Studio Code*

Visual Studio Code is the editor the team used for all Python, HTML, CSS, and JavaScript work outside of Jupyter notebooks. It was chosen for its lightweight footprint, its excellent integrated terminal, and its rich extension ecosystem — Python language support, Pylance type checking, GitLens for source-control

---

insight, Prettier for HTML/CSS formatting, and a Live Server extension for browser hot-reload during front-end development. In CloudAtk-ML, VS Code hosted the entire Flask application: `app.py` and the supporting Python modules, the sixteen Jinja2 HTML templates under `/templates`, the static CSS and JavaScript assets under `/static`, and the integration test scripts.

#### 13.2.4 Flask



*Figure 13.4: Flask Server*

Flask is the lightweight Python web framework that serves CloudAtk-ML to the browser. Flask was chosen over heavier alternatives (Django, FastAPI) because it offers a minimal core, explicit routing, and a Jinja2 templating engine that fits naturally with a project of this scope. In this system, Flask hosts every public and authenticated endpoint — `/`, `/login`, `/register`, `/dashboard`, `/predict`, `/analyze`, `/history`, `/threat_intelligence`, `/attack_map`, `/monitor`, `/security_score`, `/attack_encyclopedia`, `/settings` — and also exposes the JSON-returning prediction endpoint that the front-end JavaScript calls to drive live detection. Flask's session management, request validation hooks, and built-in development server were all used during the construction of the application.

#### 13.2.5 MySQL



*Figure 13.5: MySQL Database*

MySQL is the relational database management system that persists CloudAtk-ML's user accounts, prediction history, alerts, and configuration settings. MySQL was selected for its maturity, its SQL standard compliance, its strong performance on the read-heavy workloads typical of dashboard applications, and its straightforward integration with Python through the pymysql and SQLAlchemy connectors. The database schema follows third normal form and includes indexes on the columns most frequently used as filter predicates by the Attack History and Threat Intelligence pages — timestamp, attack\_type, severity, and status. All credentials are stored as salted hashes, never as plaintext.

13.2.6 Draw.io



Figure 13.6: Draw.io Diagramming Tool

Draw.io (now also known as diagrams.net) is the free, browser-based diagramming tool the team used to design and refine the UML diagrams of CloudAtk-ML — the use-case diagram, the class diagram, the sequence diagram, the activity diagram, and the system-architecture diagram presented in earlier chapters. Draw.io was preferred over heavier commercial alternatives because it requires no installation, runs entirely in the browser, supports collaborative editing through linked Google Drive files, and exports diagrams to PNG, SVG, and PDF formats compatible with this report. Every behavioural and structural model in this document was authored in Draw.io and then exported for inclusion.

Table 13.1: Tools and Technologies Used

| Tool               | Category               | Role in CloudAtk-ML                                            |
|--------------------|------------------------|----------------------------------------------------------------|
| Python 3.10+       | Programming language   | Implements all backend logic, ML pipeline, and Flask routes.   |
| Anaconda Navigator | Environment management | Manages the isolated Python environment and Jupyter notebooks. |

| Tool                      | Category           | Role in CloudAtk-ML                                                                                                      |
|---------------------------|--------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Visual Studio Code</b> | IDE / Code editor  | Hosts the Flask code base, templates, and static assets.                                                                 |
| <b>Flask</b>              | Web framework      | Serves all routes and the prediction REST endpoint.                                                                      |
| <b>MySQL</b>              | Database           | Persists users, predictions, alerts, and settings.                                                                       |
| <b>Draw.io</b>            | Diagramming        | Authors all UML and architecture diagrams in this report.                                                                |
| <b>scikit-learn</b>       | ML library         | Provides StandardScaler, LabelEncoder, Logistic Regression, Decision Tree, Naive Bayes, Random Forest, Isolation Forest. |
| <b>XGBoost</b>            | ML library         | Implements the production gradient-boosting classifier.                                                                  |
| <b>SHAP</b>               | Explainability     | Generates per-feature attributions for the Deep Analysis page.                                                           |
| <b>Bootstrap 5</b>        | Frontend framework | Provides the responsive UI grid and components.                                                                          |
| <b>Chart.js</b>           | Visualization      | Renders the dashboard, history, and forecast charts.                                                                     |
| <b>pandas / NumPy</b>     | Data manipulation  | Loads and reshapes the UNSW-NB15 dataset.                                                                                |

## 13.3 Creating the User Interface

### 13.3.1 Designing the User Interface

The user interface of CloudAtk-ML was designed before any HTML was written. The team produced low-fidelity wireframes for each of the sixteen pages on paper, refined them into Draw.io storyboards, and reviewed them with the supervisor before moving to code. This upfront design pass surfaced layout and navigation issues at a stage when changes were cheap, and it produced a single shared design language that was carried through every subsequent template. By the time HTML coding began, the team knew exactly what each page would look like, what controls it would expose, and how it would link to its neighbours.

### 13.3.2 Using Standards

Every HTML page in CloudAtk-ML is HTML5 compliant and validates cleanly against the W3C Markup Validation Service. The pages use semantic tags (header, nav, main, section, article, footer) rather than non-semantic divs wherever possible, both for accessibility and for clarity. Forms include proper

---

label-for-input associations so that screen readers can announce field names. ARIA attributes are added where the visual semantics need to be communicated explicitly, such as on the live attack feed and the loading spinners. Following these standards ensured that the UI is accessible to a wider range of users and is more likely to remain functional as browsers evolve.

### **13.3.3 Using Cascading Style Sheets**

All visual styling lives in external CSS files rather than inline in the HTML. The base styling comes from Bootstrap 5, which provides the responsive grid, typography, and component library. On top of Bootstrap, a single project-specific stylesheet defines the CloudAtk-ML colour palette (navy primary, teal accent), the dashboard card styles, the severity badges, and a small handful of custom animations for the live attack map. Keeping styling external rather than inline made it possible to change the visual language of the entire application by editing one file, and it kept the HTML templates readable.

## **13.4 Coding Issues**

### **13.4.1 Using Client-Side Validation**

Forms in CloudAtk-ML perform client-side validation in the browser before submitting data to the server. HTML5 validation attributes (required, pattern, minlength, maxlength) catch the most common errors at zero JavaScript cost, and a small amount of custom JavaScript handles the more complex constraints — for example, ensuring that the start of a date range is before the end. Client-side validation gives the user immediate feedback and reduces server load, but it is never trusted on its own: every submitted value is re-validated on the server. This belt-and-braces approach is essential for security.

### **13.4.2 Variable and Function Naming Convention**

All Python code follows PEP-8 conventions: snake\_case for variables and functions, PascalCase for classes, UPPER\_SNAKE\_CASE for constants. Names are descriptive — predict\_attack rather than pa\_attack\_history\_query rather than ahq — even at the cost of being long, because we found that descriptive names paid back many times over during code review. JavaScript code follows the analogous camelCase convention. The team used the Pylance language server in VS Code to flag PEP-8 deviations as they were typed, so that style violations were corrected before they entered the repository.



---

### 13.4.3 Adopting a Generic Approach

Where similar logic was needed in multiple places, the team factored it into a single generic function rather than duplicating it. The most striking example is the model-comparison code: the same function evaluates Logistic Regression, Naive Bayes, Decision Tree, Random Forest, XGBoost, and Isolation Forest, parameterized only by the model object passed in. This generic approach paid off twice — once in clarity (the comparison logic lives in exactly one place) and once in extensibility (adding a seventh model would require only a one-line change).

### 13.4.4 Keeping Class Members Private

Python does not enforce private members at the language level, but the team followed the convention that any attribute or method intended for internal use is prefixed with a single underscore. This discipline made the public API of each class explicit and let reviewers spot encapsulation violations easily. For example, the `DetectionEngine` class exposes only `predict()` and `classify()` publicly; its `_scaler`, `_label_encoder`, and `_models` attributes are all marked private and are never accessed from outside the class.

### 13.4.5 Garbage Collection

Python's reference-counted garbage collector handles memory management automatically, but a few patterns can defeat it. The team avoided long-lived global state, closed file handles and database cursors deterministically with the `with`-statement, and explicitly released NumPy arrays and pandas DataFrames after use in long-running scripts. The trained models were loaded once at server startup and held in module-level globals for the lifetime of the Flask process; this is not a memory leak, but it is a deliberate policy choice that keeps prediction latency low at the cost of a few hundred megabytes of resident memory.

### 13.4.6 Handling Errors

Every Flask endpoint is wrapped in a `try/except` that catches expected and unexpected errors and returns a meaningful response. Expected errors (missing form field, invalid date, unauthorized access) return a 400-class status code with a friendly message. Unexpected errors (database connection failure, model deserialization error) return a 500 status code with a sanitized message and are also logged in detail to the server log so that operators can diagnose them. The user is never shown a Python stack trace; they see a clean error page with a link back to the dashboard.

---

### 13.4.7 Re-using Code

Common functionality was placed in helper modules rather than copy-pasted across the application. Authentication helpers live in `auth.py`, database helpers in `db.py`, prediction helpers in `ml.py`, and PDF/CSV export helpers in `exports.py`. Whenever a developer needed authentication, prediction, or export functionality, they imported it from the appropriate helper module rather than re-implementing it. This re-use kept the codebase compact and made changes easy: fixing a bug in `db.py`'s connection pool fixed it everywhere.

## 13.5 Database Issues

### 13.5.1 Choosing a suitable MySQL database type

MySQL supports two main storage engines: MyISAM and InnoDB. The team chose InnoDB because it supports transactions and foreign-key constraints — both essential for a system that writes user accounts, predictions, and alerts atomically. MyISAM would have been faster for purely read-heavy workloads but would have allowed inconsistencies between related tables. The performance penalty of InnoDB on this workload was negligible because the read path is dominated by indexed queries that MySQL handles well regardless of the engine.

### 13.5.2 Consistency between Tables and Classes

Every persistent class identified in Chapter 8 maps to exactly one MySQL table. Column names match attribute names, column types match attribute types where the database type system allows, and foreign-key relationships in the schema mirror association relationships in the class diagram. This consistency makes the database easy to reason about: a developer who has read the class diagram already knows what the tables look like, and vice versa. The mapping was checked at the end of the integration phase to catch any drift between the two representations, and a small handful of mismatches were corrected before final delivery.

### 13.5.3 User Authentication

User passwords are never stored in plaintext. At registration time, the password is salted with a random per-user salt and hashed using the `scrypt` algorithm provided by `werkzeug.security`. At login time, the submitted password is hashed with the same salt and compared against the stored hash using a constant-time comparison function to prevent timing attacks. Sessions use server-side storage (a signed cookie

---

referring to a server-side session record) so that no sensitive data is ever held in the cookie itself. Sessions expire after a configurable period of inactivity, defaulting to 30 minutes.

## 13.6 Object-Oriented Code Segment Examples

The following code segments illustrate the style and quality of the CloudAtk-ML codebase. They are not exhaustive — the full implementation contains several thousand lines of Python and HTML — but they are representative of the conventions described above.

### Example 1: The `DetectionEngine` class

This class encapsulates model loading, feature preprocessing, and prediction. Note the underscore-prefixed private attributes, the descriptive method names, and the explicit error handling. The constructor loads all six models from disk at startup; the `predict` method accepts a feature dictionary, applies the scaler and label encoder, and returns a prediction tuple. The class is exposed to Flask routes through a single module-level instance, so the cost of model loading is paid only once.

### Example 2: The `/predict` route

The Flask `/predict` endpoint validates the incoming JSON, calls the `DetectionEngine`, persists the result to MySQL, and returns the verdict and confidence to the browser. The route is short — fewer than thirty lines — because the heavy lifting lives in the `DetectionEngine` and `Database` helper classes. The route demonstrates the team's preferred separation of concerns: routes are thin and delegate to domain classes.

### Example 3: The `Database` persistence helper

All SQL queries live in this single module so that the rest of the application never sees raw SQL. Queries use parameterized statements rather than string concatenation, which protects against SQL injection by construction. Connections are obtained from a pool and returned promptly, and the `with`-statement ensures that they are closed even if an exception is raised.

---

## 14. Evaluation and Testing

### 14.1 Overview of Testing

CloudAtk-ML was tested at four levels: unit testing (individual functions and classes), integration testing (interactions between modules), system testing (end-to-end behaviour), and user acceptance testing (validation by real users). Each level addresses a different failure mode and a different stakeholder concern. Unit tests catch bugs in individual functions early and cheaply; integration tests catch bugs in the seams between modules; system tests catch bugs in the full deployed pipeline; user acceptance tests validate that the deployed system actually meets the needs of its intended users. The four sections below describe each level with concrete test cases drawn from the project's test suite.

Test infrastructure: unit and integration tests were written using the pytest framework. System tests were run manually against a local deployment of the Flask server. User acceptance tests were performed with a small group of cybersecurity-track peers acting as proxy users. Test results were tracked in a shared spreadsheet, and any failing test was raised as an issue and fixed before the test was considered passed.

### 14.2 Class Test Specification (Unit Testing)

Unit tests target the smallest testable parts of the system — individual functions and methods on individual classes. Two of the most security-critical modules are presented below: the Authentication module and the Prediction module. Each module has its own unit-test table listing the test ID, the case description, the expected output, and the observed result.

| Test ID    | Test Case Description                                      | Expected Output                                         | Result |
|------------|------------------------------------------------------------|---------------------------------------------------------|--------|
| UT-AUTH-01 | Register new user with valid username, email, and password | User row inserted; password stored as script hash       | PASS   |
| UT-AUTH-02 | Register with duplicate username                           | Reject; descriptive error message                       | PASS   |
| UT-AUTH-03 | Register with invalid email format                         | Reject; client-side and server-side validation triggers | PASS   |
| UT-AUTH-04 | Login with correct credentials                             | Session created; redirect to dashboard                  | PASS   |
| UT-AUTH-05 | Login with incorrect password                              | Reject; generic error (no enumeration)                  | PASS   |

| Test ID    | Test Case Description                          | Expected Output                          | Result |
|------------|------------------------------------------------|------------------------------------------|--------|
| UT-AUTH-06 | Login with non-existent username               | Reject; generic error (no enumeration)   | PASS   |
| UT-AUTH-07 | Access protected page without session          | Redirect to /login                       | PASS   |
| UT-AUTH-08 | Session expires after 30 minutes of inactivity | User redirected to login on next request | PASS   |

*Table 14.1: Unit Test Cases — Authentication Module*

| Test ID    | Test Case Description                                          | Expected Output                                          | Result |
|------------|----------------------------------------------------------------|----------------------------------------------------------|--------|
| UT-PRED-01 | Submit valid feature vector to predict()                       | Return (label, confidence) tuple                         | PASS   |
| UT-PRED-02 | Submit feature vector with missing column                      | Raise ValueError with field name                         | PASS   |
| UT-PRED-03 | Submit feature vector with non-numeric value in numeric column | Raise TypeError with field name                          | PASS   |
| UT-PRED-04 | Verify StandardScaler applied before classifier                | Scaled features have zero mean and unit variance         | PASS   |
| UT-PRED-05 | Verify LabelEncoder maps known categorical value               | Returns expected encoded integer                         | PASS   |
| UT-PRED-06 | Verify XGBoost is the production model                         | Returned predictor is XGBClassifier instance             | PASS   |
| UT-PRED-07 | Confidence value is in [0, 1]                                  | Returned confidence $\geq 0$ and $\leq 1$                | PASS   |
| UT-PRED-08 | Prediction latency under 500 ms                                | Average over 100 calls $< 500$ ms                        | PASS   |
| UT-PRED-09 | compare_models() returns metrics for all six models            | Six rows of (accuracy, precision, recall, F1)            | PASS   |
| UT-PRED-10 | SHAP values sum to model output (TreeSHAP property)            | $ \text{sum}(\text{SHAP}) - \text{logit}  < 1\text{e-}6$ | PASS   |

*Table 14.2: Unit Test Cases — Prediction Module*

---

## 14.3 Integration Testing

Integration testing exercises the seams between modules. The most important integrations in CloudAtk-ML are between Flask and the database, between Flask and the DetectionEngine, and between the DetectionEngine and the SHAP explainer. Each integration is covered by at least one explicit test.

| Test ID | Integration Scenario                                                | Expected Behaviour                          | Result |
|---------|---------------------------------------------------------------------|---------------------------------------------|--------|
| IT-01   | Flask /register endpoint writes to MySQL users table                | New row visible in users table              | PASS   |
| IT-02   | Flask /predict endpoint invokes DetectionEngine and persists result | New row in predictions table; correct label | PASS   |
| IT-03   | DetectionEngine.predict() invokes XGBoost via scikit-learn API      | Returns predicted class and confidence      | PASS   |
| IT-04   | DetectionEngine.shap_values() invokes TreeSHAP on trained XGBoost   | Returns array of per-feature impacts        | PASS   |
| IT-05   | Attack History page filter triggers indexed MySQL query             | Query plan uses timestamp/severity index    | PASS   |
| IT-06   | Threat Intelligence page aggregates predictions from MySQL          | Aggregated counts match raw row counts      | PASS   |
| IT-07   | Logout invalidates the server-side session                          | Subsequent request redirects to /login      | PASS   |

*Table 14.3: Integration Test Cases*

## 14.4 System Testing

System testing exercises the full deployed application from the browser. Test cases were written as scripted scenarios that a tester would perform end-to-end. Each scenario corresponds to one or more functional requirements from Chapter 6.

| Test ID | End-to-End Scenario                                                          | Expected Outcome                                     | Result |
|---------|------------------------------------------------------------------------------|------------------------------------------------------|--------|
| ST-01   | Register, login, view dashboard, logout                                      | All four steps succeed; KPI cards render             | PASS   |
| ST-02   | Submit a flow on Live Detection page; verify it appears on Attack History    | Prediction visible within 5 s with correct timestamp | PASS   |
| ST-03   | Open Deep Analysis for a prediction; verify SHAP and model-comparison panels | Both panels render with correct data                 | PASS   |
| ST-04   | Filter Attack History by severity; export CSV                                | Filtered set downloaded as CSV                       | PASS   |
| ST-05   | Open Threat Intelligence page; verify 7-day forecast renders                 | Risk forecast chart populated with data              | PASS   |
| ST-06   | Open Global Attack Map; verify live feed updates                             | Arcs animate; feed entries appear                    | PASS   |
| ST-07   | Real-time Monitor shows live status indicators                               | All status panes update on auto-refresh              | PASS   |
| ST-08   | Concurrent users do not see each other's data                                | Per-user data isolation confirmed                    | PASS   |

*Table 14.4: System Test Cases*

---

## 15. Conclusions and Future Enhancements

### 15.1 Conclusions

This project set out to address a concrete and urgent problem: traditional, signature-based defenses cannot keep up with the pace and sophistication of attacks against modern cloud environments. CloudAtk-ML — the system designed and implemented in this report — demonstrates that machine-learning-based attack detection, when paired with a deployable web application and explainability tooling, is a viable answer to that problem.

The empirical contribution of this work is the systematic comparison of six classification models — Logistic Regression, Gaussian Naive Bayes, Decision Tree, Random Forest, XGBoost, and Isolation Forest — on the UNSW-NB15 benchmark, all under identical preprocessing. Random Forest reached 96.03% accuracy and XGBoost reached 95.94% accuracy, with Decision Tree close behind at 94.91% and Logistic Regression at 93.58%. Confusion-matrix analysis confirmed that the supervised models maintain low false-negative rates against the attack class, with XGBoost missing only 894 out of 11,200 attacks in the test set. The XGBoost model was selected for production because of its training efficiency, native SHAP support, and proven robustness on imbalanced cybersecurity data.

The engineering contribution of this work is the integration of these results into a complete Flask web application. The CloudAtk-ML system delivers a full security-administrator experience: registration, login, a key-performance-indicator dashboard, live detection, deep analysis with SHAP feature impact, threat intelligence forecasting, a global threat map, an attack encyclopedia, attack history with PDF/CSV export, a real-time monitor, a security-score view, and configurable settings. The system is supported by a normalized MySQL schema and a clean four-tier architecture that makes future extension straightforward, and the testing suite reported in Chapter 14 provides high confidence that the system behaves as specified.

Together, the empirical and engineering contributions produce more than a model and more than a notebook: they produce a tool. CloudAtk-ML is the kind of artifact that can sit on the screen of a security-operations-center analyst — readable, interactive, explainable, and grounded in measurably strong machine-learning performance. It demonstrates, at a small but meaningful scale, that the gap between cybersecurity research and cybersecurity practice can be closed with disciplined engineering.



---

## 15.2 Future Enhancements

Several directions can extend the work. First, the system can be integrated with a real-time packet-capture layer (e.g., Zeek or Suricata) to ingest live network flows directly rather than relying on submitted feature vectors. This integration would make the system suitable for production deployment in front of live traffic rather than only for retrospective analysis.

Second, the model registry can be extended with deep-learning architectures — CNN, LSTM, or Transformer encoders — to test whether they outperform the gradient-boosted baseline on the same UNSW-NB15 evaluation. Recent literature suggests that deep-learning approaches retain value for attack categories with sequential or spatial structure, and a head-to-head comparison would be informative.

Third, the explainability layer can be deepened. The current SHAP implementation explains individual predictions; future work can produce global feature-importance reports, counter-factual explanations ("this would have been classified as normal if feature X had been below threshold Y"), and an in-page tutorial that walks new users through the SHAP visualization the first time they encounter it.

Fourth, the system can address concept drift — the gradual shift in attack patterns over time — by adding scheduled retraining and a drift-detection component that alerts the administrator when the model's calibration degrades. Production deployment will eventually surface drift, and proactive monitoring is the right way to handle it.

Fifth, the platform can integrate with external SIEM systems through standardized protocols (STIX / TAXII), allowing it to act as a feeder to Splunk, IBM QRadar, or Microsoft Sentinel deployments. This integration would let CloudAtk-ML augment rather than replace existing enterprise security infrastructure.

Finally, the front-end can grow a mobile-friendly companion application so that on-call responders can be notified and triage attacks from outside the SOC. Mobile push notifications combined with a streamlined attack-history view would allow analysts to keep watch on the system even when they are away from their desks.

---

## References

- [1] U. Oktay and O. K. Sahingoz, "Attack Types and Intrusion Detection Systems in Cloud Computing," 6th International Information Security and Cryptology Conference, 2013.
- [2] A. Ghaffa and R. Soomro, "Big Data Analysis: An Apache Spark Perspective," Global Journal of Computer Science and Technology: Software & Data Engineering, vol. 15, no. 1, 2015.
- [3] M. H. Ali and M. F. Zolkipli, "Intrusion-Detection System Based on Fast Learning Network in Cloud Computing," Advanced Science Letters, 2018.
- [4] C. Modi, D. Patel, B. Borisaniya, A. Patel, and M. Rajarajan, "A survey on security issues and solutions at different layers of Cloud computing," Journal of Supercomputing, 2013.
- [5] J. M. Fossaceca, T. A. Mazzuchi, and S. Sarkani, "MARK-ELM: Application of a novel multiple-kernel learning framework for improving the robustness of network intrusion detection," Expert Systems with Applications, 2015.
- [6] N. Chand, P. Mishra, C. R. Krishna, E. S. Pilli, and M. C. Govil, "A comparative analysis of SVM and its stacking with other classification algorithms for intrusion detection," 2016 International Conference on Advances in Computing, Communication, & Automation, 2016.
- [7] Z. He, T. Zhang, and R. B. Lee, "Machine Learning Based DDoS Attack Detection from Source Side in Cloud," 2017 IEEE International Conference on Cyber Security and Cloud Computing, 2017.
- [8] W.-L. Chu, C.-J. Lin, and K.-N. Chang, "Detection and Classification of Advanced Persistent Threats and Attacks Using the Support Vector Machine," Applied Sciences, vol. 9, no. 21, 4579, 2019.
- [9] M. Ahsan and K. E. Nygard, "Convolutional Neural Networks with LSTM for Intrusion Detection," 2020.
- [10] G. Tiwari and R. Jain, "Detecting and Classifying Incoming Traffic in a Secure Cloud Computing Environment Using Machine Learning and Deep Learning System," 2022 IEEE 7th International Conference on Smart Cloud (SmartCloud), 2022, pp. 16–21.
- [11] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive dataset for network intrusion detection systems," 2015 Military Communications and Information Systems Conference (MilCIS), 2015.

- 
- [12] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794.
- [13] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," Advances in Neural Information Processing Systems 30, 2017.
- [14] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [15] M. Grinberg, Flask Web Development: Developing Web Applications with Python, 2nd ed., O'Reilly Media, 2018.

---

## Bibliography

The following references provided general background reading that informed the project but are not directly cited in the body of the report. They are included here to help the interested reader pursue the topics covered above in greater depth.

- Anthropic Press releases on AI security, 2023–2024.
- AWS Security Best Practices, Amazon Web Services Whitepaper, 2023.
- Bishop, C. M., Pattern Recognition and Machine Learning, Springer, 2006.
- Géron, A., Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow, 2nd ed., O'Reilly Media, 2019.
- Goodfellow, I., Bengio, Y., and Courville, A., Deep Learning, MIT Press, 2016.
- ISO/IEC 27001 — Information Security Management Systems, International Organization for Standardization.
- Saudi National Cybersecurity Authority (NCA), Essential Cybersecurity Controls (ECC-1:2018).
- Saudi Vision 2030 — Digital Transformation Initiative documents, 2016–2024.
- Stallings, W., Cryptography and Network Security: Principles and Practice, 8th ed., Pearson, 2020.
- The Open Web Application Security Project (OWASP) Top 10, 2021.